

Was ein KI-Agent in sechs Stunden ohne Aufsicht geschafft hat

Ein lokal laufender Agent bekam eine Liste offener Programmieraufgaben und sechs Stunden Zeit. Kein Mensch hat eingegriffen, keine Aufgabe wurde nachgebessert. Jede Aufgabe gilt erst als gelöst, wenn ein unabhängiger Testlauf sie bestätigt — nicht, wenn der Agent das behauptet. Diese Seite zeigt alle 376 gelösten Aufgaben, jede zuerst in Klartext und dann im Detail.

376 AUFGABEN GELÖST	0 FEHLSCHLÄGE	5.341 ZEILEN CODE	52 s MEDIAN JE AUFGABE
0 MANIPULIERTE TESTS			

Die 5.341 Zeilen verteilen sich auf 1.644 in den 68 einzeln geschriebenen Aufgaben und 3.697 in den 308 Varianten. Gezählt ohne Leerzeilen und Kommentare.

Wie das abläuft

Der Prüfstand besteht aus Modulen, die nichts tun ausser einen Fehler zu werfen, und je einem Test dazu. **Der Test ist die Spezifikation** — er beschreibt, was herauskommen muss, und er ist für den Agenten tabu.

Pro Aufgabe startet ein frischer Agentendurchgang. Er liest den Ist-Zustand, ändert die eine Moduldatei und führt den Abnahmebefehl aus. Danach führt der Rahmen denselben Befehl **noch einmal selbst** aus. Nur wenn dieser zweite, unabhängige Lauf mit Exitcode 0 endet, gilt die Aufgabe als gelöst. Die Behauptung des Agenten zählt nicht.

Nach jedem Lauf wird geprüft, ob eine Testdatei verändert wurde. In diesem Lauf: **keine einzige**. Der kürzeste Weg zu einem grünen Test wäre gewesen, den Test umzuschreiben. Er hat es nicht getan.

Die 68 einzeln geschriebenen Aufgaben

Jede davon ist ein eigenes Problem — von römischen Zahlen über IBAN-Prüfziffern bis zu Graphenalgorithmen. Aufklappen zeigt die Schnittstelle, den Abnahmebefehl und den Code, den der Agent tatsächlich geschrieben hat.

001 Zahl in roemische Ziffern wandeln

58 s

Eine gewöhnliche Zahl in römische Ziffern schreiben, also 1994 zu MCMXCIV.

DER HAKEN

Die subtraktive Schreibweise. 4 ist IV und nicht IIII, 900 ist CM und nicht DCCCC. Wer stur von gross nach klein herunterzählt, produziert Zahlen, die zwar den richtigen Wert haben, aber falsch geschrieben sind.

WO DAS VORKOMMT

Filmabspänne, Uhrenzifferblätter, Gliederungen in Verträgen.

SCHNITTSTELLE `roemisch(zahl)`

ABNAHME `python -m unittest tests.test_roemisch` — nur Exitcode 0 zählt als gelöst

UMFANG 21 Zeilen

```
"""roemisch: Zahl in roemische Ziffern wandeln"""
from __future__ import annotations

def roemisch(zahl: int) -> str:
    """Wandelt 1..3999 in roemische Ziffern (Grossbuchstaben, subtraktive Schreibweise:
    4=IV, 9=IX, 40=XL, 90=XC, 400=CD, 900=CM). Ausserhalb 1..3999 ValueError."""
    if not isinstance(zahl, int) or isinstance(zahl, bool):
        raise ValueError("zahl muss eine ganze Zahl sein")
    if not 1 <= zahl <= 3999:
        raise ValueError(f"Zahl muss in 1..3999 liegen, ist {zahl}")
    tabelle = (
        (1000, "M"), (900, "CM"), (500, "D"), (400, "CD"),
        (100, "C"), (90, "XC"), (50, "L"), (40, "XL"),
        (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I"),
    )
    erg = []
    rest = zahl
    for wert, zeichen in tabelle:
        while rest >= wert:
            erg.append(zeichen)
            rest -= wert
    return "".join(erg)
```

Der umgekehrte Weg: aus MCMXCIV wieder 1994 machen.

DER HAKEN

Die Rückrichtung ist deutlich härter, weil sie auch Unsinn erkennen muss. IIII und IL sehen wie Zahlen aus, sind aber keine gültige römische Schreibweise und müssen abgelehnt werden. Genau diese Aufgabe hat Ben in der ganzen Reihe am häufigsten gestolpert.

WO DAS VORKOMMT

Alte Datenbestände einlesen, in denen Jahreszahlen römisch stehen.

SCHNITTSTELLE `roemisch_lesen(text)`

ABNAHME `python -m unittest tests.test_roemisch_lesen` – nur Exitcode 0 zählt als gelöst

UMFANG 20 Zeilen

```
"""roemisch_lesen: roemische Ziffern als Zahl lesen"""
from __future__ import annotations

import re

_WERTE = {"I": 1, "V": 5, "X": 10, "L": 50, "C": 100, "D": 500, "M": 1000}
# Roemische Zahl in strikt kanonischer Form: M* (CM|CD|D?C*) (XC|XL|L?X*) (IX|IV|I*)
_KANONISCH = re.compile(
    r"^M{0,3}"
    r"(?:CM|CD|D?C{0,3})?"
    r"(?:XC|XL|L?X{0,3})?"
    r"(?:IX|IV|I{1,3})$"
)

def roemisch_lesen(text: str) -> int:
    """Liest roemische Ziffern (Grossbuchstaben) als int. Ungueltige Eingabe: ValueError."""
    if not isinstance(text, str) or not _KANONISCH.match(text):
        raise ValueError(f"Ungueltige roemische Ziffern: {text!r}")
    total = 0
    for i, zeichen in enumerate(text):
        wert = _WERTE[zeichen]
        naechster = _WERTE.get(text[i + 1], 0) if i + 1 < len(text) else 0
        total = total - wert if wert < naechster else total + wert
    return total
```

Die Prüfziffer, mit der fast jede Kreditkartennummer sich selbst kontrolliert.

DER HAKEN

Jede zweite Ziffer wird verdoppelt, und ist das Ergebnis zweistellig, werden dessen Ziffern addiert. Ob man von links oder rechts zu zählen beginnt, entscheidet über richtig und falsch.

WO DAS VORKOMMT

Bezahlformulare erkennen damit Tippfehler, bevor überhaupt eine Bank gefragt wird.

SCHNITTSTELLE `luhn_guelting(ziffern)`

ABNAHME `python -m unittest tests.test_luhn` – nur Exitcode 0 zählt als gelöst

UMFANG 16 Zeilen

```
"""luhn: Pruefsumme nach Luhn"""
from __future__ import annotations

def luhn_guelting(ziffern: str) -> bool:
    """Prueft eine Ziffernfolge gegen die Luhn-Pruefsumme. Nur Ziffern erlaubt,
    sonst ValueError. Leere Eingabe ist ValueError."""
    if not ziffern or not ziffern.isdigit():
        raise ValueError("nur Ziffern erlaubt, Eingabe nicht leer")
    total = 0
    for i, z in enumerate(reversed(ziffern)):
        d = int(z)
        if i % 2 == 1:
            d *= 2
            if d > 9:
                d -= 9
        total += d
    return total % 10 == 0
```

Prüfen, ob eine IBAN echt sein kann.

DER HAKEN

Die ersten vier Zeichen wandern ans Ende, Buchstaben werden zu zweistelligen Zahlen, und das Ganze muss beim Teilen durch 97 den Rest 1 lassen. Die entstehende Zahl hat über dreissig Stellen — in Sprachen mit begrenzten Ganzzahlen ist das der eigentliche Stolperstein.

WO DAS VORKOMMT

Jede Überweisungsmaske prüft das, bevor sie das Geld losschickt.

SCHNITTSTELLE `iban_gueltig(iban)`

ABNAHME `python -m unittest tests.test_iban` – nur Exitcode 0 zählt als gelöst

UMFANG 12 Zeilen

```
"""iban: IBAN-Pruefziffer nach mod 97"""
from __future__ import annotations

def iban_gueltig(iban: str) -> bool:
    """Prueft eine IBAN nach mod-97-10. Leerzeichen werden ignoriert, Kleinbuchstaben
    akzeptiert. Zu kurze oder nicht alphanumerische Eingabe: ValueError."""
    sauber = iban.replace(" ", "").upper()
    if not sauber or not sauber.isalnum() or len(sauber) < 5:
        raise ValueError(f"Keine gueltige IBAN: {iban!r}")

    # Land + Praefix nach hinten schieben, A-Z zu 10-35 wandeln.
    zaiffern = "".join(
        str(ord(c) - 55) if c.isalpha() else c for c in sauber[4:] + sauber[:4]
    )
    return int(zaiffern) % 97 == 1
```

Lauf längenkodierung: aus AAAB wird A3B1, und wieder zurück.

DER HAKEN

Der Rückweg muss mehrstellige Zahlen vertragen. Steht dort A12, sind das zwölf A und nicht ein A, gefolgt von einem falsch gelesenen Rest.

WO DAS VORKOMMT

Faxgeräte, alte Bildformate, und bis heute die Komprimierung einfarbiger Flächen.

SCHNITTSTELLE `rle_kodieren(text) · rle_dekodieren(text)`

ABNAHME `python -m unittest tests.test_rle` – nur Exitcode 0 zählt als gelöst

UMFANG 38 Zeilen

```
"""rle: Lauf längenkodierung hin und zurueck"""
from __future__ import annotations

def rle_kodieren(text: str) -> str:
    """Lauf längenkodierung: jede Folge gleicher Zeichen wird zu Zeichen+Anzahl,
    auch bei Anzahl 1. 'aaabbc' -> 'a3b2c1'. Leerer Text -> leerer Text.
    rle_dekodieren kehrt das exakt um; kaputte Eingabe: ValueError."""
    if not text:
        return ""
    ausgabe = []
    i = 0
    while i < len(text):
        anfang = i
        while i + 1 < len(text) and text[i + 1] == text[i]:
            i += 1
        ausgabe.append(text[anfang] + str(i - anfang + 1))
        i += 1
    return "".join(ausgabe)

def rle_dekodieren(text: str) -> str:
    """Kehrt rle_kodieren exakt um."""
    if not text:
        return ""
    ausgabe = []
    i = 0
    while i < len(text):
        zeichen = text[i]
        if i + 1 >= len(text):
            raise ValueError("unvollstaendiger Block: " + zeichen)
        anfang = i + 1
        while anfang < len(text) and text[anfang].isdigit():
            anfang += 1
        if anfang == i + 1:
            raise ValueError("fehlt Anzahl nach: " + zeichen)
        anzahl = int(text[i + 1:anfang])
        if anzahl == 0:
            raise ValueError("Anzahl darf nicht 0 sein: " + zeichen)
        ausgabe.append(zeichen * anzahl)
        i = anfang
    return "".join(ausgabe)
```

Wie viele einzelne Änderungen braucht man, um ein Wort in ein anderes zu verwandeln.

DER HAKEN

Die naive Lösung probiert alle Möglichkeiten durch und braucht bei längeren Wörtern länger als ein Menschenleben. Es geht nur mit einer Tabelle, die Zwischenergebnisse aufhebt.

WO DAS VORKOMMT

Meintest du ...? in jeder Suchmaske, Rechtschreibkorrektur, Abgleich von Kundendaten.

SCHNITTSTELLE `levenshtein(links, rechts)`

ABNAHME `python -m unittest tests.test_levenshtein` – nur Exitcode 0 zählt als gelöst

UMFANG 18 Zeilen

```
"""levenshtein: Editierabstand"""
from __future__ import annotations

def levenshtein(links: str, rechts: str) -> int:
    """Editierabstand mit Einfuegen, Loeschen und Ersetzen zu je Kosten 1."""
    if not links:
        return len(rechts)
    if not rechts:
        return len(links)

    vor = list(range(len(rechts) + 1))
    for i, zl in enumerate(links, start=1):
        jetzt = [i]
        for j, zr in enumerate(rechts, start=1):
            kosten = 0 if zl == zr else 1
            jetzt.append(
                min(vor[j] + 1, jetzt[j - 1] + 1, vor[j - 1] + kosten)
            )
        vor = jetzt
    return vor[-1]
```

Eine Zahl in ihre Primfaktoren zerlegen, 60 also in 2, 2, 3, 5.

DER HAKEN

Man darf nur bis zur Wurzel suchen — und muss daran denken, dass der Rest am Ende selbst noch eine Primzahl sein kann.

WO DAS VORKOMMT

Grundlage jeder Verschlüsselung, die auf grossen Zahlen beruht.

SCHNITTSTELLE `primfaktoren(zahl)`

ABNAHME `python -m unittest tests.test_primfaktoren` – nur Exitcode 0 zählt als gelöst

UMFANG 15 Zeilen

```
"""primfaktoren: Zahl in Primfaktoren zerlegen"""
from __future__ import annotations

def primfaktoren(zahl: int) -> list[int]:
    """Primfaktoren aufsteigend, mit Vielfachheit. primfaktoren(1) ist die leere Liste.
    Zahlen kleiner 1: ValueError."""
    if zahl < 1:
        raise ValueError("Zahl muss mindestens 1 sein")
    ergebnis: list[int] = []
    faktor = 2
    while zahl > 1:
        while zahl % faktor == 0:
            ergebnis.append(faktor)
            zahl //= faktor
        faktor += 1
    return ergebnis
```


Einen Bruch kürzen, aus 6/8 also 3/4 machen.

DER HAKEN

Das Vorzeichen. $-3/-4$ ist positiv, $3/-4$ gehört als $-3/4$ geschrieben. Und Null im Nenner muss sauber abgelehnt werden statt abzustürzen.

WO DAS VORKOMMT

Überall dort, wo Verhältnisse exakt bleiben müssen und Kommazahlen nicht reichen.

SCHNITTSTELLE `bruch_kuerzen(zaehler, nenner)`

ABNAHME `python -m unittest tests.test_bruch` – nur Exitcode 0 zählt als gelöst

UMFANG 21 Zeilen

```
"""bruch: Bruch kürzen mit Vorzeichenregel"""
from __future__ import annotations

def bruch_kuerzen(zaehler: int, nenner: int) -> tuple[int, int]:
    """Kuerzt einen Bruch vollstaendig. Das Vorzeichen steht immer im Zaehler,
    der Nenner ist immer positiv. bruch_kuerzen(0, 5) ist (0, 1).
    Nenner 0: ZeroDivisionError."""
    if nenner == 0:
        raise ZeroDivisionError("Nenner darf nicht null sein.")
    if zaehler == 0:
        return (0, 1)
    if (zaehler < 0) != (nenner < 0):
        zaehler = abs(zaehler)
        nenner = abs(nenner)
        zaehler = -zaehler
    else:
        zaehler = abs(zaehler)
        nenner = abs(nenner)
    a, b = abs(zaehler), nenner
    while b:
        a, b = b, a % b
    return (zaehler // a, nenner // a)
```

Die einfachste Geheimschrift: jeden Buchstaben um eine feste Zahl weiterschieben.

DER HAKEN

Alles, was kein Buchstabe ist, muss unverändert bleiben, und Gross- und Kleinschreibung dürfen nicht durcheinandergeraten. Der Ringschluss von Z zurück zu A ist die zweite Falle.

WO DAS VORKOMMT

Historisch beim Militär, heute noch als Denksportaufgabe.

SCHNITTSTELLE `caesar(text, verschiebung)`

ABNAHME `python -m unittest tests.test_caesar` – nur Exitcode 0 zählt als gelöst

UMFANG 16 Zeilen

```
"""caesar: Verschiebechiffre mit Zeichenerhalt"""
from __future__ import annotations

def caesar(text: str, verschiebung: int) -> str:
    """Caesar-Verschiebung nur fuer a-z und A-Z, Gross/Klein bleibt erhalten,
    alles andere unveraendert. Negative und grosse Verschiebungen sind erlaubt."""
    out = []
    for ch in text:
        if "a" <= ch <= "z":
            base = ord("a")
        elif "A" <= ch <= "Z":
            base = ord("A")
        else:
            out.append(ch)
            continue
        out.append(chr(base + (ord(ch) - base + verschiebung) % 26))
    return "".join(out)
```

Zählen, wie viele Einsen in der Binärdarstellung einer Zahl stehen.

DER HAKEN

Klingt trivial, ist es auch — bis negative Zahlen kommen. Dann hängt die Antwort davon ab, wie man sie überhaupt darstellt.

WO DAS VORKOMMT

Fehlerkorrektur, Kompression, Prüfsummen in Netzwerkprotokollen.

SCHNITTSTELLE `bitzaehler(zahl)`

ABNAHME `python -m unittest tests.test_bitzaehler` – nur Exitcode 0
zählt als gelöst

UMFANG 7 Zeilen

```
"""bitzaehler: gesetzte Bits zaehlen"""
from __future__ import annotations

def bitzaehler(zahl: int) -> int:
    """Zählt gesetzte Bits einer nicht negativen ganzen Zahl. Negativ: ValueError."""
    if zahl < 0:
        raise ValueError("zahl darf nicht negativ sein")
    return bin(zahl).count("1")
```

Sortieren, wie ein Mensch es erwartet: Datei2 vor Datei10, nicht umgekehrt.

DER HAKEN

Die übliche Sortierung vergleicht Zeichen für Zeichen, und da steht 1 vor 2. Man muss die Ziffernblöcke erkennen und als Zahl vergleichen, den Rest aber als Text.

WO DAS VORKOMMT

Jeder Dateimanager, jede Bildergalerie, jede Kapitelliste.

SCHNITTSTELLE `_natuerlicher_schluessel(zeichenkette)` ·
`natuerlich_sortieren(werte)`

ABNAHME `python -m unittest tests.test_natuerlich` – nur Exitcode 0
zählt als gelöst

UMFANG 21 Zeilen

```
"""natuerlich: natuerliche Sortierung von Zeichenketten"""
from __future__ import annotations

import re

_ZIFFERREIHE = re.compile(r"(\d+)")

def _natuerlicher_schluessel(zeichenkette: str) -> tuple:
    """Zerlegt eine Zeichenkette in Text- und Zahlenbestandteile.

    Ziffernfolgen werden als Zahlen in den Schluessel aufgenommen, damit
    'a2' vor 'a10' steht und nicht 'a10' vor 'a2' (wie beim lexicographischen
    Vergleich von Text). Nicht-Ziffern bleiben Text.
    """
    teile = []
    for stueck in _ZIFFERREIHE.split(zeichenkette):
        if stueck.isdigit():
            teile.append((1, int(stueck)))
        elif stueck:
            teile.append((0, stueck))
    return tuple(teile)

def natuerlich_sortieren(werte: list[str]) -> list[str]:
    """Sortiert Zeichenketten natuerlich: Ziffernfolgen zaehlen als Zahl, nicht als Text.
    'a2' steht vor 'a10'. Sortierung ist stabil, die Eingabeliste bleibt unveraendert."""
    return sorted(werte, key=_natuerlicher_schluessel)
```

Eine Sekundenzahl als Zeitspanne darstellen, 3661 also als 1:01:01.

DER HAKEN

Die führenden Nullen und die Frage, ob der Stundenteil bei null überhaupt erscheinen soll.

WO DAS VORKOMMT

Videospieler, Stoppuhren, Fortschrittsanzeigen.

SCHNITTSTELLE `zeitspanne_formatieren(sekunden)`

ABNAHME `python -m unittest tests.test_zeitspanne` – nur Exitcode 0 zählt als gelöst

UMFANG 16 Zeilen

```
"""zeitspanne: Sekunden lesbar formatieren"""
from __future__ import annotations

def zeitspanne_formatieren(sekunden: int) -> str:
    """Formatiert Sekunden als 'Xd Xh Xm Xs'. Führende Nulleinheiten entfallen,
    innere nicht. 0 ergibt genau '0s'. Negativ: ValueError."""
    if sekunden < 0:
        raise ValueError("Sekunden darf nicht negativ sein.")
    if sekunden == 0:
        return "0s"
    tage, rest = divmod(sekunden, 86400)
    stunden, rest = divmod(rest, 3600)
    minuten, sek_rest = divmod(rest, 60)
    teile: list[tuple[int, str]] = [(tage, "d"), (stunden, "h"), (minuten, "m"), (sek_rest, "s")]
    while teile and teile[0][0] == 0:
        teile.pop(0)
    return " ".join(f"{wert}{einheit}" for wert, einheit in teile)
```

Zu einem Datum den Wochentag ausrechnen.

DER HAKEN

Schaltjahre folgen keiner einfachen Regel: alle vier Jahre, ausser alle hundert, ausser alle vierhundert. 1900 war keines, 2000 schon.

WO DAS VORKOMMT

Kalender, Terminplanung, Schichtpläne.

SCHNITTSTELLE `wochentag(jahr, monat, tag)`

ABNAHME `python -m unittest tests.test_wochentag` – nur Exitcode 0 zählt als gelöst

UMFANG 10 Zeilen

```
"""wochentag: deutscher Wochentag zu einem Datum"""
from __future__ import annotations

import datetime

TAGE = ("Montag", "Dienstag", "Mittwoch", "Donnerstag",
        "Freitag", "Samstag", "Sonntag")

def wochentag(jahr: int, monat: int, tag: int) -> str:
    """Liefert den deutschen Wochentagsnamen, etwa 'Montag'. Ungueltiges Datum: ValueError.
    Die Berechnung darf datetime benutzen."""
    datum = datetime.date(jahr, monat, tag) # ValueError bei ungueltigem Datum
    return TAGE[datum.weekday()]
```

Eine Tabellenzeile in ihre Felder zerlegen.

DER HAKEN

Anführungszeichen. Steht das Trennzeichen innerhalb von Anführungszeichen, trennt es nicht — und zwei Anführungszeichen hintereinander bedeuten ein wörtliches. Wer einfach am Komma trennt, zerreisst jede Adresse.

WO DAS VORKOMMT

Jeder Datenimport aus Excel, jeder Export aus einem Warenwirtschaftssystem.

SCHNITTSTELLE `_pruefen(zeile) · csv_zerlegen(zeile)`

ABNAHME `python -m unittest tests.test_csvzeile` — nur Exitcode 0 zählt als gelöst

UMFANG 29 Zeilen

```
"""csvzeile: CSV-Zeile nach RFC 4180 zerlegen"""
from __future__ import annotations

import csv
from io import StringIO

def _pruefen(zeile: str) -> None:
    """Scannt die rohe Zeile und wirft ValueError bei unbalancierten Anfuehrungszeichen."""
    in_quotes = False
    i, n = 0, len(zeile)
    while i < n:
        c = zeile[i]
        if in_quotes:
            if c == "'":
                if i + 1 < n and zeile[i + 1] == "'":
                    i += 2
                    continue
                in_quotes = False
            i += 1
            continue
        if c == "'":
            in_quotes = True
            i += 1
        if in_quotes:
            raise ValueError(f"unbalancierte Anfuehrungszeichen in {zeile!r}")

def csv_zerlegen(zeile: str) -> list[str]:
    """Zerlegt eine CSV-Zeile nach RFC 4180: Komma trennt, Anfuehrungszeichen schuetzen
    Kommas, doppelte Anfuehrungszeichen im Feld stehen fuer eines. Leere Zeile
    ergibt ein leeres Feld. Unbalancierte Anfuehrungszeichen: ValueError."""
    _pruefen(zeile)
    return next(csv.reader(StringIO(zeile)), [""])
```

In einer sortierten Liste einen Wert finden, indem man immer halbiert.

DER HAKEN

Die berüchtigtste Aufgabe der Informatik. Ein Standardwerk brauchte Jahrzehnte, bis die abgedruckte Fassung stimmte; die Fehler stecken immer in den Rändern.

WO DAS VORKOMMT

Datenbank-Indizes, Autovervollständigung, jede Suche in sortierten Daten.

SCHNITTSTELLE `binaersuche(werte, gesucht)`

ABNAHME `python -m unittest tests.test_binaersuche` – nur Exitcode 0 zählt als gelöst

UMFANG 15 Zeilen

```
"""binaersuche: binaere Suche mit kleinstem Index"""
from __future__ import annotations

def binaersuche(werte: list[int], gesucht: int) -> int:
    """Binaere Suche in einer aufsteigend sortierten Liste. Liefert den Index oder -1.
    Bei Dubletten den kleinsten passenden Index."""
    links, rechts = 0, len(werte)
    while links < rechts:
        mitte = (links + rechts) // 2
        if werte[mitte] < gesucht:
            links = mitte + 1
        else:
            rechts = mitte
    if links < len(werte) and werte[links] == gesucht:
        return links
    return -1
```


Prüfen, ob Klammern korrekt verschachtelt sind.

DER HAKEN

Zählen reicht nicht. `([])` hat gleich viele öffnende und schliessende Klammern und ist trotzdem falsch. Man braucht einen Stapel, der sich merkt, was zuletzt geöffnet wurde.

WO DAS VORKOMMT

Jeder Editor, der Klammerfehler anzeigt, jeder Übersetzer einer Programmiersprache.

SCHNITTSTELLE `klammern_gueltig(text)`

ABNAHME `python -m unittest tests.test_klammern` – nur Exitcode 0 zählt als gelöst

UMFANG 15 Zeilen

```
"""klammern: Klammerschachtelung pruefen"""
from __future__ import annotations

PAIRS = {"(": ")", "[": "]", "{": "}"}
OPENERS = set(PAIRS.values())

def klammern_gueltig(text: str) -> bool:
    """Prueft (), [] und {} auf korrekte Schachtelung. Andere Zeichen werden ignoriert.
    Leerer Text ist gueltig."""
    stack: list[str] = []
    for zeichen in text:
        if zeichen in PAIRS:
            if not stack or stack.pop() != PAIRS[zeichen]:
                return False
        elif zeichen in OPENERS:
            stack.append(zeichen)
    return not stack
```

Die berühmte Zahlenfolge: gerade halbieren, ungerade mal drei plus eins — bis man bei 1 landet.

DER HAKEN

Kein Mensch hat bisher bewiesen, dass sie immer bei 1 endet. Programmieren lässt sie sich trotzdem, man muss nur die Schritte richtig zählen.

WO DAS VORKOMMT

Ein offenes mathematisches Problem, das seit 1937 ungelöst ist.

SCHNITTSTELLE `kollatz_laenge(start)`

ABNAHME `python -m unittest tests.test_kollatz` – nur Exitcode 0 zählt als gelöst

UMFANG 12 Zeilen

```
"""kollatz: Laenge der Collatz-Folge"""
from __future__ import annotations

def kollatz_laenge(start: int) -> int:
    """Laenge der Collatz-Folge einschliesslich Start und der abschliessenden 1.
    kollatz_laenge(1) ist 1. Werte kleiner 1: ValueError."""
    if start < 1:
        raise ValueError("start muss eine ganze Zahl >= 1 sein")
    laenge = 1
    while start != 1:
        start = start // 2 if start % 2 == 0 else 3 * start + 1
        laenge += 1
    return laenge
```

Grundrechenarten auf Zahlentabellen.

DER HAKEN

Die Ränder: leere Tabellen, unterschiedlich lange Zeilen. Der Hauptfall ist in fünf Zeilen erledigt, die Prüfungen drumherum brauchen mehr.

WO DAS VORKOMMT

Bildbearbeitung, Statistik, Physiksimulation.

SCHNITTSTELLE `transponieren(matrix)`

ABNAHME `python -m unittest tests.test_matrix` – nur Exitcode 0 zählt als gelöst

UMFANG 12 Zeilen

```
"""matrix: Matrix transponieren"""
from __future__ import annotations

def transponieren(matrix: list[list[int]]) -> list[list[int]]:
    """Transponiert eine rechteckige Matrix. Leere Matrix ergibt leere Liste.
    Ungleich lange Zeilen: ValueError."""
    if not matrix:
        return []
    breite = len(matrix[0])
    for zeile in matrix:
        if len(zeile) != breite:
            raise ValueError("Zeilen haben unterschiedliche Laengen.")
    return [[zeile[i] for zeile in matrix] for i in range(breite)]
```

Eine Zahl in eine andere Basis schreiben, dezimal 255 also als ff im Hexadezimalsystem.

DER HAKEN

Die Null und die negativen Zahlen. Genau hier hat Ben in einem anderen Lauf eine Endlosschleife gebaut: er negierte die Zahl bedingungslos, und die Schleife kam nie zum Ende.

WO DAS VORKOMMT

Farbwerte im Web, Speicheradressen, Prüfsummen.

SCHNITTSTELLE `basis_wandeln(zahl, basis)`

ABNAHME `python -m unittest tests.test_basis_wandeln` – nur Exitcode 0 zählt als gelöst

UMFANG 21 Zeilen

```
"""basis_wandeln: Zahl in eine Basis von 2 bis 36 wandeln"""
from __future__ import annotations

_ZIFFERN = "0123456789abcdefghijklmnopqrstuvwxyz"

def basis_wandeln(zahl: int, basis: int) -> str:
    """Wandelt eine ganze Zahl in die angegebene Basis (2..36).

    Ziffern sind 0-9 und Kleinbuchstaben a-z. Negative Zahlen bekommen ein
    fuehrendes Minus, die Null ergibt "0". Basis ausserhalb 2..36: ValueError.
    """
    if not 2 <= basis <= 36:
        raise ValueError(f"Basis muss zwischen 2 und 36 liegen, ist {basis}")
    if zahl == 0:
        return "0"
    negativ = zahl < 0
    wert = -zahl if negativ else zahl
    ziffern = []
    while wert:
        wert, rest = divmod(wert, basis)
        ziffern.append(_ZIFFERN[rest])
    if negativ:
        ziffern.append("-")
    return "".join(reversed(ziffern))
```

Der Rückweg: ff wieder als 255 lesen.

DER HAKEN

Ungültige Ziffern müssen auffallen. In Basis 8 gibt es keine 9, und das darf nicht stillschweigend durchgehen.

WO DAS VORKOMMT

Konfigurationsdateien, in denen Rechte oktal stehen.

SCHNITTSTELLE `basis_lesen(text, basis)`

ABNAHME `python -m unittest tests.test_basis_lesen` – nur Exitcode 0 zählt als gelöst

UMFANG 17 Zeilen

```
"""basis_lesen: Zeichenkette aus einer Basis von 2 bis 36 lesen"""
from __future__ import annotations

def basis_lesen(text: str, basis: int) -> int:
    """Liest eine Zeichenkette in der angegebenen Basis (2..36) als int.

    Gross- und Kleinschreibung ist egal, ein führendes Minus ist erlaubt.
    Leere Eingabe, ungültige Ziffer fuer diese Basis oder eine Basis
    ausserhalb 2..36 werfen ValueError.
    """
    if not isinstance(basis, int) or isinstance(basis, bool):
        raise ValueError(f"Basis muss ein Integer sein, nicht {basis!r}")
    if not 2 <= basis <= 36:
        raise ValueError(f"Basis muss zwischen 2 und 36 liegen, ist {basis}")
    if not isinstance(text, str):
        raise ValueError(f"Eingabe muss eine Zeichenkette sein, nicht {text!r}")
    if not text.strip():
        raise ValueError("Leere Eingabe")
    return int(text, basis)
```

021 **groesster gemeinsamer Teiler und kleinstes gemeinsames Vielfaches**

45 s

Grösster gemeinsamer Teiler und kleinstes gemeinsames Vielfaches.

DER HAKEN

Die Null. Der Teiler von 0 und 5 ist 5, aber das Vielfache mit einer Null ist wieder null — und das Standardverfahren teilt an dieser Stelle gern durch null.

WO DAS VORKOMMT

Brüche kürzen, Zahnräder auslegen, Takte synchronisieren.

SCHNITTSTELLE `ggt(a, b) · kgv(a, b)`

ABNAHME `python -m unittest tests.test_ggt_kgv` — nur Exitcode 0 zählt als gelöst

UMFANG 16 Zeilen

```
"""ggt_kgv: groesster gemeinsamer Teiler und kleinstes gemeinsames Vielfaches"""
from __future__ import annotations

def ggt(a: int, b: int) -> int:
    """Groesster gemeinsamer Teiler, immer nicht negativ. ggt(0, 0) ist 0."""
    a, b = abs(a), abs(b)
    while b:
        a, b = b, a % b
    return a

def kgv(a: int, b: int) -> int:
    """Kleinstes gemeinsames Vielfaches, immer nicht negativ.

    Ist einer der Werte 0, ist das Ergebnis 0.
    """
    a, b = abs(a), abs(b)
    if a == 0 or b == 0:
        return 0
    return a // ggt(a, b) * b
```

Die Folge, in der jede Zahl die Summe ihrer beiden Vorgänger ist.

DER HAKEN

Die naive rekursive Fassung rechnet dieselben Werte millionenfach neu. Für die zwanzigste Zahl reicht das, für die fünfzigste nicht mehr.

WO DAS VORKOMMT

Das Standardbeispiel dafür, wie man Zwischenergebnisse aufhebt.

SCHNITTSTELLE `fib(n) · folge(n)`

ABNAHME `python -m unittest tests.test_fibonacci` – nur Exitcode 0 zählt als gelöst

UMFANG 22 Zeilen

```
"""fibonacci: n-te Fibonacci-Zahl und Folge"""
from __future__ import annotations

def fib(n: int) -> int:
    """n-te Fibonacci-Zahl mit fib(0) = 0 und fib(1) = 1.

    Negatives n wirft ValueError. Muss auch fuer n = 10000 zuegig laufen.
    """
    if n < 0:
        raise ValueError("n muss nicht negativ sein")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

def folge(n: int) -> list[int]:
    """Die ersten n Fibonacci-Zahlen ab fib(0). n = 0 ergibt eine leere Liste."""
    if n < 0:
        raise ValueError("n muss nicht negativ sein")
    ergebnis = []
    a, b = 0, 1
    for _ in range(n):
        ergebnis.append(a)
        a, b = b, a + b
    return ergebnis
```

Die Ziffern einer Zahl addieren, so lange, bis nur eine Ziffer übrig bleibt.

DER HAKEN

Die wiederholte Anwendung. Und: negative Zahlen und die Null brauchen eine Entscheidung, keine Ausnahme.

WO DAS VORKOMMT

Prüfziffern, Teilbarkeitsregeln, Zahlenrätsel.

SCHNITTSTELLE `quersumme(zahl) · digitale_wurzel(zahl)`

ABNAHME `python -m unittest tests.test_quersumme` – nur Exitcode 0 zählt als gelöst

UMFANG 11 Zeilen

```
"""quersumme: Quersumme und digitale Wurzel"""
from __future__ import annotations

def quersumme(zahl: int) -> int:
    """Summe der Ziffern des Betrags. quersumme(-123) ist 6."""
    return sum(int(ziffer) for ziffer in str(abs(zahl)))

def digitale_wurzel(zahl: int) -> int:
    """Wiederholte Quersumme, bis nur noch eine Ziffer bleibt."""
    wert = abs(zahl)
    while wert >= 10:
        wert = quersumme(wert)
    return wert
```


Alle Teiler einer Zahl addieren.

DER HAKEN

Ob die Zahl selbst mitzählt, ändert alles. Ohne sie geht es um vollkommene Zahlen, mit ihr um etwas anderes — und die Aufgabe verlangt eine bestimmte Lesart.

WO DAS VORKOMMT

Zahlentheorie, und historisch der Ursprung der vollkommenen Zahlen.

SCHNITTSTELLE `echte_teiler(zahl) · einordnen(zahl)`

ABNAHME `python -m unittest tests.test_teilersumme` – nur Exitcode 0 zählt als gelöst

UMFANG 19 Zeilen

```
"""teilersumme: echte Teiler summieren und die Zahl einordnen"""
from __future__ import annotations

def echte_teiler(zahl: int) -> list[int]:
    """Aufsteigend sortierte echte Teiler ohne die Zahl selbst.

    echte_teiler(1) ist eine leere Liste. Zahlen kleiner 1 werfen ValueError.
    """
    if zahl < 1:
        raise ValueError(f"zahl muss >= 1 sein, ist {zahl}")
    return [teiler for teiler in range(1, zahl) if zahl % teiler == 0]

def einordnen(zahl: int) -> str:
    """Vergleicht die Summe der echten Teiler mit der Zahl.

    Groesser ist "abundant", gleich ist "vollkommen", kleiner ist "defizient".
    """
    summe = sum(echte_teiler(zahl))
    if summe == zahl:
        return "vollkommen"
    if summe > zahl:
        return "abundant"
    return "defizient"
```

Das Zahlendreieck, in dem jede Zahl die Summe der beiden über ihr ist.

DER HAKEN

Die Ränder sind immer eins, und die Zeilennummerierung beginnt je nach Lehrbuch bei null oder eins. Ein Fehler um eins verschiebt das ganze Dreieck.

WO DAS VORKOMMT

Wahrscheinlichkeitsrechnung, Binomialkoeffizienten, Lotto.

SCHNITTSTELLE `zeile(n) · dreieck(n)`

ABNAHME `python -m unittest tests.test_pascal` – nur Exitcode 0 zählt als gelöst

UMFANG 15 Zeilen

```
"""pascal: Zeile des Pascalschen Dreiecks"""
from __future__ import annotations

def zeile(n: int) -> list[int]:
    """n-te Zeile des Pascalschen Dreiecks, beginnend bei zeile(0) = [1].

    Negatives n wirft ValueError.
    """
    if n < 0:
        raise ValueError(f"n muss nichtnegativ sein, ist {n}")
    zeile_werte = [1]
    for _ in range(n):
        zeile_werte = [1] + [a + b for a, b in zip(zeile_werte, zeile_werte[1:])] + [1]
    return zeile_werte

def dreieck(n: int) -> list[list[int]]:
    """Die ersten n Zeilen. dreieck(0) ist eine leere Liste."""
    return [zeile(i) for i in range(n)]
```

Alle Primzahlen bis zu einer Grenze finden, nach dem Sieb des Eratosthenes.

DER HAKEN

Man beginnt beim Streichen erst beim Quadrat der Primzahl, nicht beim Doppelten — sonst arbeitet man um ein Vielfaches länger als nötig.

WO DAS VORKOMMT

Kryptografie, und seit 250 vor Christus praktisch unverändert.

SCHNITTSTELLE `primzahlen_bis(n)`

ABNAHME `python -m unittest tests.test_sieb` – nur Exitcode 0 zählt als gelöst

UMFANG 18 Zeilen

```
"""sieb: Primzahlen bis n mit dem Sieb des Eratosthenes"""
from __future__ import annotations

def primzahlen_bis(n: int) -> list[int]:
    """Alle Primzahlen kleiner oder gleich n, aufsteigend.

    Fuer n kleiner 2 eine leere Liste. Muss auch fuer n = 200000 zuegig laufen,
    eine Probedivision je Zahl ist dafuer zu langsam.
    """
    if n < 2:
        return []
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    k = 2
    while k * k <= n:
        if is_prime[k]:
            for multiple in range(k * k, n + 1, k):
                is_prime[multiple] = False
            k += 1
    return [zahl for zahl in range(2, n + 1) if is_prime[zahl]]
```

Zählen, an wie vielen Stellen sich zwei gleich lange Zeichenketten unterscheiden.

DER HAKEN

Ungleich lange Eingaben sind kein Ergebnis, sondern ein Fehler. Wer sie stillschweigend abschneidet, liefert eine Zahl, die nichts bedeutet.

WO DAS VORKOMMT

Fehlerkorrektur bei Funkübertragung, DNA-Vergleiche.

SCHNITTSTELLE `hamming_abstand(links, rechts)`

ABNAHME `python -m unittest tests.test_hamming` – nur Exitcode 0 zählt als gelöst

UMFANG 11 Zeilen

```
"""hamming: Hamming-Abstand zweier gleich langer Zeichenketten"""
from __future__ import annotations

def hamming_abstand(links: str, rechts: str) -> int:
    """Anzahl der Stellen, an denen sich beide Zeichenketten unterscheiden.

    Unterschiedliche Laenge wirft ValueError. Zwei leere Zeichenketten ergeben 0.
    """
    if len(links) != len(rechts):
        raise ValueError(
            f"Eingaben haben unterschiedliche Laenge: {len(links)} und {len(rechts)}"
        )
    return sum(1 for a, b in zip(links, rechts) if a != b)
```

Einen Betrag mit möglichst wenigen Münzen zusammenlegen.

DER HAKEN

Immer die grösste Münze zuerst zu nehmen ist bei Euro richtig, bei anderen Münzsystemen aber falsch. Die allgemeine Lösung braucht deshalb einen anderen Ansatz als die naheliegende.

WO DAS VORKOMMT

Kassensysteme, Automaten, und ein Klassiker der Optimierung.

SCHNITTSTELLE `min_muenzen(betrag, werte)`

ABNAHME `python -m unittest tests.test_muenzen` – nur Exitcode 0 zählt als gelöst

UMFANG 25 Zeilen

```
"""muenzen: minimale Anzahl Muenzen fuer einen Betrag"""
from __future__ import annotations

def min_muenzen(betrag: int, werte: list[int]) -> int:
    """Kleinste Anzahl Muenzen, die den Betrag exakt ergeben.

    Jede Muenze ist unbegrenzt verfuegbar. Ein gieriges Verfahren genuegt nicht:
    bei werte = [1, 3, 4] und betrag = 6 lautet die Antwort 2 (3+3), nicht 3.
    Betrag 0 ergibt 0. Negativer Betrag oder ein nicht darstellbarer Betrag
    wirft ValueError.
    """
    if betrag < 0:
        raise ValueError("Betrag muss nichtnegativ sein.")
    if betrag == 0:
        return 0
    if not werte or min(werte) <= 0:
        raise ValueError("Muenswerte muessen positiv sein.")

    # dp[b] = kleinste Muenzenzahl fuer Betrag b; inf = nicht erreichbar
    inf = betrag + 1 # mehr als betrag Muenzen sind nie noetig
    dp: list[int] = [inf] * (betrag + 1)
    dp[0] = 0
    for b in range(1, betrag + 1):
        for w in werte:
            if w <= b and dp[b - w] + 1 < dp[b]:
                dp[b] = dp[b - w] + 1

    if dp[betrag] == inf:
        raise ValueError(f"Betrag {betrag} ist mit {werte} nicht darstellbar.")
    return dp[betrag]
```

Prüfen, ob etwas vorwärts wie rückwärts gleich gelesen wird.

DER HAKEN

Was zählt als Zeichen? Leerzeichen, Satzzeichen und Grossschreibung müssen erst weggeräumt werden, sonst ist kaum ein Satz ein Palindrom.

WO DAS VORKOMMT

Sprachspiele — und als Muster in der DNA-Analyse.

SCHNITTSTELLE `normalisieren(text) · ist_palindrom(text)`

ABNAHME `python -m unittest tests.test_palindrom` – nur Exitcode 0
zählt als gelöst

UMFANG 11 Zeilen

```
"""palindrom: Palindrom pruefen und Zeichen normalisieren"""
from __future__ import annotations

def normalisieren(text: str) -> str:
    """Nur Buchstaben und Ziffern, alles klein geschrieben."""
    return "".join(ch.lower() for ch in text if ch.isalnum())

def ist_palindrom(text: str) -> bool:
    """Prueft auf Palindrom, ignoriert Schreibweise und Satzzeichen.

    Die leere Zeichenkette gilt als Palindrom.
    """
    norm = normalisieren(text)
    return norm == norm[::-1]
```

Prüfen, ob zwei Wörter dieselben Buchstaben in anderer Reihenfolge verwenden.

DER HAKEN

Es reicht nicht, die Buchstaben zu vergleichen — es kommt auf die Anzahl an. aab und abb bestehen aus denselben Buchstaben und sind trotzdem kein Anagramm.

WO DAS VORKOMMT

Worträtsel, und als schneller Vorfilter beim Datenabgleich.

SCHNITTSTELLE `_schluessel(text) · ist_anagramm(links, rechts)`
`· gruppieren(woerter)`

ABNAHME `python -m unittest tests.test_anagramm` — nur Exitcode 0 zählt als gelöst

UMFANG 19 Zeilen

```
"""anagramm: Anagramme erkennen und gruppieren"""
from __future__ import annotations

def _schluessel(text: str) -> str:
    """Sortiert Buchstaben und Ziffern, ignoriert alles andere und Gross/Klein."""
    return "".join(sorted(ch.lower() for ch in text if ch.isalnum()))

def ist_anagramm(links: str, rechts: str) -> bool:
    """Prueft, ob beide dieselben Zeichen in anderer Reihenfolge enthalten.

    Schreibweise und alle Zeichen ausser Buchstaben und Ziffern werden ignoriert.
    """
    return _schluessel(links) == _schluessel(rechts)

def gruppieren(woerter: list[str]) -> list[list[str]]:
    """Fasst Anagramme zu Gruppen zusammen.

    Die Gruppen werden nach ihrem ersten Wort sortiert, innerhalb einer Gruppe
    bleibt die Eingabereihenfolge erhalten.
    """
    gruppen: dict[str, list[str]] = {}
    for wort in woerter:
        gruppen.setdefault(_schluessel(wort), []).append(wort)
    return [gruppe for _, gruppe in sorted(gruppen.items(), key=lambda kv: kv[1][0])]
```

Zählen, welches Wort in einem Text am häufigsten vorkommt.

DER HAKEN

Der Gleichstand. Kommen zwei Wörter gleich oft vor, muss eine Regel entscheiden — sonst liefert dasselbe Programm bei jedem Lauf ein anderes Ergebnis.

WO DAS VORKOMMT

Suchmaschinen, Textanalyse, Schlagwortwolken.

SCHNITTSTELLE `zaehlen(text) · haeufigste(text, n)`

ABNAHME `python -m unittest tests.test_wortfrequenz` – nur Exitcode 0 zählt als gelöst

UMFANG 23 Zeilen

```
"""wortfrequenz: haeufigste Woerter eines Textes"""
from __future__ import annotations

def zaehlen(text: str) -> dict[str, int]:
    """Zaehlt Woerter, klein geschrieben.

    Ein Wort ist eine zusammenhaengende Folge aus Buchstaben und Ziffern; alles
    andere trennt.
    """
    import re
    zähler: dict[str, int] = {}
    for wort in re.findall(r"[0-9A-Za-z]+", text):
        wort = wort.lower()
        zähler[word] = zähler.get(word, 0) + 1
    return zähler

def haeufigste(text: str, n: int) -> list[tuple[str, int]]:
    """Die n haeufigsten Woerter als Paare (wort, anzahl).

    Sortiert nach Haeufigkeit absteigend, bei Gleichstand alphabetisch
    aufsteigend. Negatives n wirft ValueError.
    """
    if n < 0:
        raise ValueError("n muss nicht negativ sein")
    zähler = zaehlen(text)
    sortiert = sorted(zähler.items(), key=lambda paare: (-paare[1], paare[0]))
    return sortiert[:n]
```


Einen Text auf eine feste Zeilenbreite umbrechen, ohne Wörter zu zerreißen.

DER HAKEN

Wörter, die allein schon länger sind als die Zeile. Und die Frage, wohin das Leerzeichen am Umbruch gehört.

WO DAS VORKOMMT

Jeder Texteditor, jede Konsolenausgabe, jede E-Mail im Nur-Text-Format.

SCHNITTSTELLE `umbrechen(text, breite)`

ABNAHME `python -m unittest tests.test_umbruch` – nur Exitcode 0 zählt als gelöst

UMFANG 29 Zeilen

```
def umbrechen(text: str, breite: int) -> list[str]:
    if breite < 1:
        raise ValueError("breite muss >= 1 sein")

    worte = text.split()
    if not worte:
        return []

    zeilen: list[str] = []
    aktuelle: list[str] = []
    laenge = 0

    for wort in worte:
        if len(wort) > breite:
            # Wort laenger als die Breite: steht ungeteilt in einer
            # eigenen Zeile. Der Rest der laufenden Zeile wird zuerst
            # geschlossen, danach folgt das Wort als eigene Zeile.
            if aktuelle:
                zeilen.append(" ".join(aktuelle))
                aktuelle = []
                laenge = 0
            zeilen.append(wort)
        elif laenge == 0:
            aktuelle.append(wort)
            laenge = len(wort)
        elif laenge + 1 + len(wort) <= breite:
            aktuelle.append(wort)
            laenge += 1 + len(wort)
        else:
            zeilen.append(" ".join(aktuelle))
            aktuelle = [wort]
            laenge = len(wort)

    if aktuelle:
        zeilen.append(" ".join(aktuelle))
    return zeilen
```

Einen Text mit Zeilennummern versehen.

DER HAKEN

Die Ausrichtung. Bei über neun Zeilen muss die Nummerierung rechtsbündig stehen, sonst verrutscht der ganze Text.

WO DAS VORKOMMT

Fehlermeldungen von Übersetzern, Codeanzeigen, Protokolle.

SCHNITTSTELLE `nummerieren(text, start)`

ABNAHME `python -m unittest tests.test_zeilennummern` – nur Exitcode 0 zählt als gelöst

UMFANG 18 Zeilen

```
"""zeilennummern: Zeilen rechtsbündig nummerieren"""
from __future__ import annotations

def nummerieren(text: str, start: int = 1) -> str:
    """Stellt jeder Zeile ihre rechtsbündige Nummer und ": " voran.

    Die Breite richtet sich nach der groessten vorkommenden Nummer. Leerer Text
    ergibt eine leere Zeichenkette; ein abschliessender Zeilenumbruch erzeugt
    keine zusaetzliche Zeile.
    """
    if not text:
        return ""
    zeilen = text.split("\n")
    if zeilen and zeilen[-1] == "":
        zeilen.pop()
    letzte = start + len(zeilen) - 1
    breite = len(str(letzte))
    return "\n".join(
        f"{n:>{breite}}: {zeile}" for n, zeile in enumerate(zeilen, start=start)
    )
```

Platzhalter in einer Vorlage durch echte Werte ersetzen.

DER HAKEN

Fehlende Werte. Bleibt der Platzhalter stehen, wird er zu einem Fehler, oder er wird zu einer leeren Stelle — eines von beidem muss festgelegt sein.

WO DAS VORKOMMT

Serienbriefe, Rechnungsvorlagen, Bestätigungsmails.

SCHNITTSTELLE `fuellen(vorlage, werte)`

ABNAHME `python -m unittest tests.test_template_fuellen` – nur Exitcode 0 zählt als gelöst

UMFANG 40 Zeilen

```
"""template_fuellen: Platzhalter in einer Vorlage ersetzen"""
from __future__ import annotations

def fuellen(vorlage: str, werte: dict[str, str]) -> str:
    """Ersetzt {name} durch werte[name].

    "{{" und "}}" stehen fuer eine einzelne geschweifte Klammer. Ein unbekannter
    Name wirft KeyError, eine nicht geschlossene Klammer ValueError.
    """
    ergebnis: list[str] = []
    i = 0
    n = len(vorlage)

    while i < n:
        zeichen = vorlage[i]
        if zeichen == "{":
            if i + 1 < n and vorlage[i + 1] == "{":
                ergebnis.append("{")
                i += 2
                continue
            start = i + 1
            end = vorlage.find("}", start)
            if end == -1:
                raise ValueError(
                    "nicht geschlossene Klammer in {}".format(vorlage[i:])
                )
            name = vorlage[start:end]
            if name not in werte:
                raise KeyError(name)
            ergebnis.append(werte[name])
            i = end + 1
        elif zeichen == "}":
            if i + 1 < n and vorlage[i + 1] == "}":
                ergebnis.append("}")
                i += 2
                continue
            raise ValueError(
                "nicht geschlossene Klammer in {}".format(vorlage[i:])
            )
        else:
            ergebnis.append(zeichen)
            i += 1

    return "".join(ergebnis)
```

Prüfen, ob ein Dateiname zu einem Muster wie *.txt passt.

DER HAKEN

Der Stern darf beliebig viel verschlucken, auch nichts. Sobald mehrere Sterne im Muster stehen, reicht ein einfacher Vergleich nicht mehr.

WO DAS VORKOMMT

Jeder Dateisuchbefehl, jede .gitignore, jede Backup-Regel.

```
SCHNITTSTELLE  passt(name, muster) · _pruefe(name, muster, i, j) ·  
_klasse_kommt(zeichen, muster, start, ende, verneint)
```

ABNAHME python -m unittest tests.test_glob_passt – nur Exitcode 0
zählt als gelöst

UMFANG 58 Zeilen

```

"""glob_passt: Glob-Muster gegen einen Namen pruefen"""
from __future__ import annotations

def passt(name: str, muster: str) -> bool:
    """Prueft einen Namen gegen ein Glob-Muster.

    "*" steht fuer beliebig viele Zeichen (auch keines), "?" fuer genau eines.
    Zeichenklassen sind [abc], Bereiche [a-z] und die Verneinung [!abc].
    Eine nicht geschlossene Klasse gilt als gewoehnliche eckige Klammer.
    """
    return _pruefe(name, muster, 0, 0)

def _pruefe(name: str, muster: str, i: int, j: int) -> bool:
    if j == len(muster):
        return i == len(name)
    zeichen = muster[j]
    if zeichen == "*":
        if _pruefe(name, muster, i, j + 1):
            return True
        if i < len(name):
            return _pruefe(name, muster, i + 1, j)
        return False
    if zeichen == "[":
        k = j + 1
        verneint = k < len(muster) and muster[k] == "!"
        if verneint:
            k += 1
        if k < len(muster) and muster[k] == "]":
            k += 1
        schliessen = k
        while schliessen < len(muster) and muster[schliessen] != "]":
            schliessen += 1
        if schliessen == len(muster):
            # Nicht geschlossene Klasse: als gewoehnliche "[" behandeln.
            if i < len(name) and name[i] == "[":
                return _pruefe(name, muster, i + 1, j + 1)
            return False
        if i >= len(name):
            return False
        return _klasse_kommt(name[i], muster, k, schliessen, verneint) and _pruefe(
            name, muster, i + 1, schliessen + 1
        )
    if zeichen == "?":
        if i < len(name):
            return _pruefe(name, muster, i + 1, j + 1)
        return False
    if i < len(name) and name[i] == zeichen:
        return _pruefe(name, muster, i + 1, j + 1)
    return False

def _klasse_kommt(zeichen: str, muster: str, start: int, ende: int, verneint: bool) -> bool:
    # ende ist die Position des schliessenden "]"
    gefunden = False
    k = start
    while k <= ende:
        if k + 2 <= ende and muster[k + 1] == "-":
            if muster[k] <= zeichen <= muster[k + 2]:
                gefunden = True
                k += 3
                continue
        if zeichen == muster[k]:
            gefunden = True
            k += 1
    return not gefunden if verneint else gefunden

```

Einen Dateipfad aufräumen: Punkte und doppelte Schrägstriche entfernen, .. eine Ebene hoch.

DER HAKEN

Was passiert, wenn .. über die Wurzel hinausgeht? Diese Frage muss beantwortet werden, sonst entsteht genau hier die Lücke, über die Angreifer aus einem Verzeichnis ausbrechen.

WO DAS VORKOMMT

Jeder Webserver, jedes Betriebssystem — und eine der häufigsten Sicherheitslücken überhaupt.

SCHNITTSTELLE `normalisieren(pfad)`

ABNAHME `python -m unittest tests.test_pfad_normalisieren` — nur Exitcode 0 zählt als gelöst

UMFANG 29 Zeilen

```
"""pfad_normalisieren: POSIX-Pfad zusammenfassen"""
from __future__ import annotations

def normalisieren(pfad: str) -> str:
    """Fasst einen POSIX-Pfad zusammen.

    Doppelte Schraegstriche und "." fallen weg, ".." wird aufgeloeset. Bei einem
    absoluten Pfad verschwindet ein ".." oberhalb der Wurzel; bei einem relativen
    Pfad bleibt es erhalten. Der leere Pfad ergibt ".".
    """
    absolut = pfad.startswith("/")
    teile = [t for t in pfad.split("/") if t not in ("", ".")]

    tiefe: list[str] = []
    oben = 0 # aufgeloesete "../" oberhalb des Ausgangspunkts

    for teil in teile:
        if teil == "..":
            if tiefe:
                tiefe.pop()
            elif absolut:
                pass # verschwindet oberhalb der Wurzel
            else:
                oben += 1
        else:
            tiefe.append(teil)

    if absolut:
        return "/" + "/".join(tiefe)
    if tiefe:
        return "/".join([".."] * oben + tiefe)
    if oben:
        return "../" * oben
    return "."
```

Den Teil einer Webadresse hinter dem Fragezeichen in seine Bestandteile zerlegen.

DER HAKEN

Derselbe Name kann mehrfach vorkommen, Werte können fehlen, und Sonderzeichen stehen kodiert da. Jeder dieser drei Fälle wird gern vergessen.

WO DAS VORKOMMT

Jede Webanwendung, bei jedem Seitenaufruf.

SCHNITTSTELLE `entschluesseln(text) · zerlegen(abfrage)`

ABNAHME `python -m unittest tests.test_query_string` – nur Exitcode 0 zählt als gelöst

UMFANG 49 Zeilen

```

"""query_string: Abfrageteil einer URL zerlegen"""
from __future__ import annotations

_HEX = "0123456789ABCDEFabcdef"

def entschluesseln(text: str) -> str:
    """Loest "+" als Leerzeichen und %XX als Zeichen auf.

    Ein unvollstaendiges oder ungueltiges %XX wirft ValueError.
    """
    ausgabe = []
    i = 0
    while i < len(text):
        zeichen = text[i]
        if zeichen == "+":
            ausgabe.append(" ")
            i += 1
        elif zeichen == "%":
            if i + 2 >= len(text) + 1 and False:
                pass
            if i + 2 >= len(text):
                raise ValueError("unvollstaendige Prozentkodierung: %r" % text[i:])
            eins = text[i + 1]
            zwei = text[i + 2]
            if eins not in _HEX or zwei not in _HEX:
                raise ValueError("ungueltige Prozentkodierung: %r" % text[i:])
            ausgabe.append(chr(int(eins + zwei, 16)))
            i += 3
        else:
            ausgabe.append(zeichen)
            i += 1
    return "".join(ausgabe)

def zerlegen(abfrage: str) -> dict[str, list[str]]:
    """Zerlegt einen Abfrageteil in Schluessel und Werteliste.

    Mehrfach vorkommende Schluessel sammeln ihre Werte in Reihenfolge. Ein
    Schluessel ohne "=" bekommt den leeren Wert. Leere Abschnitte werden
    uebersprungen. Ein fuehrendes "?" wird abgeschnitten.
    """
    if abfrage.startswith("?"):
        abfrage = abfrage[1:]
    erg: dict[str, list[str]] = {}
    for abschnitt in abfrage.split("&"):
        if not abschnitt:
            continue
        if "=" in abschnitt:
            schluessel, wert = abschnitt.split("=", 1)
        else:
            schluessel, wert = abschnitt, ""
        key = entschluesseln(schluessel)
        value = entschluesseln(wert)
        erg.setdefault(key, []).append(value)
    return erg

```


Eine einfache Konfigurationsdatei mit Abschnitten und Schlüsseln einlesen.

DER HAKEN

Kommentare, Leerzeilen und Leerzeichen um das Gleichheitszeichen. Und die Frage, was mit einem Schlüssel geschieht, der zweimal vorkommt.

WO DAS VORKOMMT

Windows seit den Neunzigern, und bis heute unzählige Programme.

SCHNITTSTELLE `lesen(text)`

ABNAHME `python -m unittest tests.test_ini_lesen` – nur Exitcode 0 zählt als gelöst

UMFANG 29 Zeilen

```
"""ini_lesen: einfaches INI-Format zerlegen"""
from __future__ import annotations

def lesen(text: str) -> dict[str, dict[str, str]]:
    """Zerlegt INI-Text in Abschnitte und Zuweisungen.

    "[name]" beginnt einen Abschnitt, "schluessel=wert" ist eine Zuweisung.
    Leerraum um Namen und Werte wird entfernt. Zeilen, die nach dem Entfernen von
    Leerraum mit ";" oder "#" beginnen, sind Kommentare; leere Zeilen werden
    uebersprungen. Schluesssel vor dem ersten Abschnitt landen unter "".
    Eine spaetere Zuweisung ueberschreibt eine fruehere. Eine Zeile, die weder
    Abschnitt noch Zuweisung ist, wirft ValueError.
    """
    ergebnis: dict[str, dict[str, str]] = {}
    aktueller: str = ""

    for zeile in text.splitlines():
        zeile = zeile.strip()
        if not zeile:
            continue
        if zeile.startswith(("@", ";", "#")):
            continue
        if zeile.startswith("[") and zeile.endswith("]"):
            aktueller = zeile[1:-1].strip()
            ergebnis.setdefault(aktueller, {})
            continue
        if "=" in zeile:
            schluessel, wert = zeile.split("=", 1)
            ergebnis.setdefault(aktueller, {})[schluessel.strip()] = wert.strip()
            continue
        raise ValueError(f"Keine gültige INI-Zeile: {zeile!r}")

    return ergebnis
```

Feststellen, ob ein Jahr ein Schaltjahr ist.

DER HAKEN

Die Dreifachregel. Durch vier teilbar ja, aber durch hundert nein, ausser durch vierhundert. Wer nur die erste Regel kennt, liegt in einem von hundert Jahren falsch.

WO DAS VORKOMMT

Jede Datumsberechnung — und die Ursache mehrerer realer Softwareausfälle.

SCHNITTSTELLE `ist_schaltjahr(jahr) · tage_im_monat(jahr, monat)`

ABNAHME `python -m unittest tests.test_schaltjahr` – nur Exitcode 0 zählt als gelöst

UMFANG 14 Zeilen

```
"""schaltjahr: Schaltjahr erkennen und Monatslaenge bestimmen"""
from __future__ import annotations

def ist_schaltjahr(jahr: int) -> bool:
    """Gregorianische Regel: durch 4, ausser durch 100, ausser durch 400."""
    return jahr % 400 == 0 or (jahr % 4 == 0 and jahr % 100 != 0)

def tage_im_monat(jahr: int, monat: int) -> int:
    """Anzahl Tage des Monats. Monat ausserhalb 1..12 wirft ValueError."""
    if not 1 <= monat <= 12:
        raise ValueError(f"Monat muss 1..12 sein, nicht {monat}")
    if monat == 2:
        return 29 if ist_schaltjahr(jahr) else 28
    if monat in (4, 6, 9, 11):
        return 30
    return 31
```

Die Anzahl Tage zwischen zwei Daten.

DER HAKEN

Monate haben verschiedene Längen, der Februar hängt vom Schaltjahr ab, und die Reihenfolge der beiden Daten darf keine Rolle spielen.

WO DAS VORKOMMT

Zinsberechnung, Mietverträge, Fristenkontrolle.

SCHNITTSTELLE `lesen(text) · differenz(von, bis)`

ABNAHME `python -m unittest tests.test_datum_differenz` – nur Exitcode

0 zählt als gelöst

UMFANG 11 Zeilen

```
"""datum_differenz: Tage zwischen zwei Datumsangaben"""
from __future__ import annotations

import datetime

def lesen(text: str) -> datetime.date:
    """Liest ein Datum im Format JJJJ-MM-TT.

    Ungültiges Format oder unmögliches Datum wie 2023-02-30 wirft ValueError.
    """
    return datetime.date.fromisoformat(text.strip())

def differenz(von: str, bis: str) -> int:
    """Anzahl Tage von "von" bis "bis". Negativ, wenn "bis" frueher liegt."""
    return (lesen(bis) - lesen(von)).days
```

Die Kalenderwoche nach ISO-Norm zu einem Datum bestimmen.

DER HAKEN

Der Jahreswechsel. Der 1. Januar kann zur letzten Woche des Vorjahres gehören, und der 31. Dezember zur ersten des Folgejahres. Die Norm ist eindeutig, aber alles andere als naheliegend.

WO DAS VORKOMMT

Produktionsplanung, Schichtpläne, Lieferterminzusagen.

SCHNITTSTELLE `kalenderwoche(text)`

ABNAHME `python -m unittest tests.test_kalenderwoche` – nur Exitcode 0 zählt als gelöst

UMFANG 12 Zeilen

```
"""kalenderwoche: ISO-Kalenderwoche eines Datums"""
from __future__ import annotations

import datetime

def kalenderwoche(text: str) -> tuple[int, int]:
    """ISO-Jahr und ISO-Woche als Paar (jahr, woche).

    Nach ISO 8601 kann der 1. Januar noch zur letzten Woche des Vorjahres
    gehoeren und der 31. Dezember schon zur ersten Woche des Folgejahres.
    Eingabeformat ist JJJJ-MM-TT, sonst ValueError.
    """
    datum = datetime.date.fromisoformat(text)
    jahr, woche, _tag = datum.isocalendar()
    return (jahr, woche)
```

Eine Angabe wie 1h30m als Sekundenzahl verstehen.

DER HAKEN

Beliebige Reihenfolge und weggelassene Teile. 90m ist gültig, 1h30m auch, und beides muss dasselbe ergeben.

WO DAS VORKOMMT

Konfigurationsdateien, Kommandozeilenwerkzeuge, Zeiterfassung.

SCHNITTSTELLE `dauer_lesen(text)`

ABNAHME `python -m unittest tests.test_dauer_lesen` – nur Exitcode 0 zählt als gelöst

UMFANG 26 Zeilen

```
"""dauer_lesen: Dauerangabe wie 1h30m in Sekunden wandeln"""
from __future__ import annotations

import re

_SEKUNDEN = {"d": 86400, "h": 3600, "m": 60, "s": 1}
_STUECK = re.compile(r"(\d+)([dhms])")

def dauer_lesen(text: str) -> int:
    """Wandelt eine Angabe wie "1h30m" oder "2d" in Sekunden.

    Einheiten sind d (Tag), h (Stunde), m (Minute) und s (Sekunde), in
    beliebiger Reihenfolge, jede hoechstens einmal. Leerraum am Rand ist
    erlaubt. Leere Angabe, unbekannte Einheit, fehlende Zahl vor einer
    Einheit, doppelte Einheit oder eine Zahl ohne Einheit werfen ValueError.
    """
    text = text.strip()
    if not text:
        raise ValueError("leere Dauerangabe")

    match = re.fullmatch(r"(?:\d+[dhms])+", text)
    if not match:
        raise ValueError(f"ungueltige Dauerangabe: {text!r}")

    total = 0
    gesehen = set()
    for zahl, einheit in _STUECK.findall(text):
        if einheit in gesehen:
            raise ValueError(f"Einheit {einheit!r} doppelt")
        gesehen.add(einheit)
        total += int(zahl) * _SEKUNDEN[einheit]
    return total
```

Eine Bytezahl lesbar machen, 1536 also als 1,5 KiB.

DER HAKEN

Kilo ist nicht gleich Kilo. 1000 oder 1024 ergeben verschiedene Ergebnisse, und die Rundung an der Grenze zur nächsten Einheit ist die zweite Falle.

WO DAS VORKOMMT

Jeder Dateimanager, jede Downloadanzeige.

SCHNITTSTELLE `formatieren(bytes_)`

ABNAHME `python -m unittest tests.test_groesse_formatieren` – nur Exitcode 0 zählt als gelöst

UMFANG 20 Zeilen

```
"""groesse_formatieren: Bytezahl lesbar darstellen"""
from __future__ import annotations

def formatieren(bytes_: int) -> str:
    """Stellt eine Bytezahl lesbar dar.

    Stufen sind B, KB, MB, GB, TB mit dem Faktor 1024. Unter 1024 Byte lautet
    die Ausgabe ganzzahlig, zum Beispiel "512 B". Darueber wird mit genau einer
    Nachkommastelle gerundet, zum Beispiel "1.5 KB". Ab TB wird nicht weiter
    gestuft. Negative Werte werfen ValueError.
    """
    if bytes_ < 0:
        raise ValueError("Negative Werte werden nicht unterstuetzt.")

    stufen = ["B", "KB", "MB", "GB", "TB"]
    wert = float(bytes_)
    for stufe in stufen:
        if wert < 1024 or stufe == stufen[-1]:
            if stufe == "B":
                return f"{int(wert)} B"
            return f"{wert:.1f} {stufe}"
        wert /= 1024
    # Unerreicht, weil die letzte Stufe immer zurueckgibt.
    return f"{wert:.1f} {stufen[-1]}"
```

Mittelwert, Median und häufigster Wert einer Zahlenreihe.

DER HAKEN

Der Median bei gerader Anzahl ist der Durchschnitt der beiden mittleren. Und beim häufigsten Wert braucht es wieder eine Regel für den Gleichstand.

WO DAS VORKOMMT

Jede Auswertung, in der Ausreisser den Durchschnitt verfälschen würden.

SCHNITTSTELLE `median(werte) * modus(werte)`

ABNAHME `python -m unittest tests.test_median_modus` – nur Exitcode 0 zählt als gelöst

UMFANG 24 Zeilen

```
"""median_modus: Median und haeufigsten Wert bestimmen"""
from __future__ import annotations

def median(werte: list[float]) -> float:
    """Median der Werte.

    Bei gerader Anzahl das arithmetische Mittel der beiden mittleren Werte.
    Leere Liste wirft ValueError.
    """
    if not werte:
        raise ValueError("median: leere Liste")
    s = sorted(werte)
    n = len(s)
    mitte = n // 2
    if n % 2:
        return s[mitte]
    return (s[mitte - 1] + s[mitte]) / 2

def modus(werte: list[float]) -> float:
    """Haeufigster Wert; bei Gleichstand der kleinste. Leere Liste: ValueError."""
    if not werte:
        raise ValueError("modus: leere Liste")
    zaehlungen: dict[float, int] = {}
    for wert in werte:
        zaehlungen[wert] = zaehlungen.get(wert, 0) + 1
    max_haeufig = max(zaehlungen.values())
    return min(w for w, h in zaehlungen.items() if h == max_haeufig)
```

Wie stark eine Zahlenreihe um ihren Mittelwert streut.

DER HAKEN

Teilt man durch n oder durch $n-1$? Beides ist richtig, aber für verschiedene Fragen — und die Wahl ändert das Ergebnis.

WO DAS VORKOMMT

Qualitätskontrolle, Finanzrisiko, jede Messreihe.

SCHNITTSTELLE `mittelwert(werte) · varianz(werte) · streuung(werte)`

ABNAHME `python -m unittest tests.test_standardabweichung` — nur Exitcode 0 zählt als gelöst

UMFANG 17 Zeilen

```
"""standardabweichung: Mittelwert, Varianz und Streuung"""
from __future__ import annotations

def mittelwert(werte: list[float]) -> float:
    """Arithmetisches Mittel. Leere Liste wirft ValueError."""
    if not werte:
        raise ValueError("mittelwert braucht mindestens einen Wert")
    return sum(werte) / len(werte)

def varianz(werte: list[float]) -> float:
    """Stichprobenvarianz mit n-1 im Nenner. Weniger als zwei Werte: ValueError."""
    n = len(werte)
    if n < 2:
        raise ValueError("varianz braucht mindestens zwei Werte")
    m = mittelwert(werte)
    return sum((w - m) ** 2 for w in werte) / (n - 1)

def streuung(werte: list[float]) -> float:
    """Standardabweichung als Wurzel der Stichprobenvarianz."""
    return varianz(werte) ** 0.5
```


Den gleitenden Durchschnitt über ein Fenster berechnen.

DER HAKEN

Der Anfang. Solange das Fenster noch nicht voll ist, muss entschieden werden, ob man kürzer mittelt oder gar nichts ausgibt.

WO DAS VORKOMMT

Börsenkurse, Sensordaten, Wettervorhersagen.

SCHNITTSTELLE `gleitmittel(werte, fenster)`

ABNAHME `python -m unittest tests.test_gleitmittel` – nur Exitcode 0 zählt als gelöst

UMFANG 12 Zeilen

```
"""gleitmittel: gleitender Durchschnitt ueber ein Fenster"""
from __future__ import annotations

def gleitmittel(werte: list[float], fenster: int) -> list[float]:
    """Gleitender Durchschnitt.

    Bei n Werten und Fenster k entstehen genau n-k+1 Ergebnisse. Fenster kleiner
    1 oder groesser als die Liste wirft ValueError, ebenso eine leere Liste.
    """
    if fenster < 1:
        raise ValueError("Fenster muss mindestens 1 sein.")
    if fenster > len(werte):
        raise ValueError("Fenster darf nicht groesser als die Liste sein.")

    return [sum(werte[i : i + fenster]) / fenster for i in range(len(werte) - fenster + 1)]
```

Sich überlappende Zeiträume zu einem zusammenfassen.

DER HAKEN

Erst sortieren, dann verschmelzen — und entscheiden, ob zwei Zeiträume, die sich nur berühren, schon verschmelzen. Das ist die Familie, an der Ben dreimal eine Wiederholung brauchte.

WO DAS VORKOMMT

Kalender, Raumbellegung, Schichtplanung.

SCHNITTSTELLE `vereinigen(intervalle)`

ABNAHME `python -m unittest tests.test_intervalle_vereinigen` – nur Exitcode 0 zählt als gelöst

UMFANG 22 Zeilen

```
"""intervalle_vereinigen: ueberlappende Intervalle zusammenfassen"""
from __future__ import annotations

def vereinigen(intervalle: list[tuple[int, int]]) -> list[tuple[int, int]]:
    """Fasst ueberlappende und beruehrende Intervalle zusammen.

    Ein Intervall ist ein Paar (start, ende) mit start kleiner gleich ende.
    (1, 2) und (2, 3) beruehren sich und werden zu (1, 3). Das Ergebnis ist nach
    Start aufsteigend sortiert. Ein Intervall mit start groesser ende wirft
    ValueError.
    """
    for start, ende in intervall:
        if start > ende:
            raise ValueError(f"Intervall ({start}, {ende}): start groesser als ende")
    if not intervall:
        return []
    sortiert = sorted(intervall, key=lambda iv: iv[0])
    erg = [list(sortiert[0])]
    for start, ende in sortiert[1:]:
        if start <= erg[-1][1]:
            erg[-1][1] = max(erg[-1][1], ende)
        else:
            erg.append([start, ende])
    return [(s, e) for s, e in erg]
```

Aus 1-3,5 die Liste 1,2,3,5 machen.

DER HAKEN

Absteigende Bereiche, einzelne Zahlen und Dubletten müssen alle vernünftig behandelt werden.

WO DAS VORKOMMT

Druckdialoge, Seitenauswahl, Portlisten in Firewalls.

SCHNITTSTELLE `ausdehnen(text)`

ABNAHME `python -m unittest tests.test_bereich_ausdehnen` – nur Exitcode 0 zählt als gelöst

UMFANG 35 Zeilen

```
"""bereich_ausdehnen: Bereichsliste wie 1-3,5 zu Zahlen ausdehnen"""
from __future__ import annotations

def ausdehnen(text: str) -> list[int]:
    """Dehnt eine Angabe wie "1-3,5,7-9" zu [1, 2, 3, 5, 7, 8, 9] aus.

    Komma trennt Abschnitte, Bindestrich beschreibt einen Bereich. Leerraum um
    die Abschnitte wird entfernt. Das Ergebnis ist aufsteigend sortiert und
    enthaelt keine Dubletten. Leere Eingabe ergibt eine leere Liste. Ein
    ungueltiger Abschnitt oder ein Bereich mit Start groesser Ende wirft
    ValueError. Negative Zahlen kommen nicht vor.
    """
    text = text.strip()
    if not text:
        return []
    ergebnis: set[int] = set()
    for abschnitt in text.split(","):
        abschnitt = abschnitt.strip()
        if "-" in abschnitt:
            teile = abschnitt.split("-")
            if len(teile) != 2:
                raise ValueError(f"Ungueltiger Abschnitt: {abschnitt!r}")
            start_text, ende_text = teile
            start_text = start_text.strip()
            ende_text = ende_text.strip()
            if not start_text.isdigit() or not ende_text.isdigit():
                raise ValueError(f"Ungueltiger Abschnitt: {abschnitt!r}")
            start = int(start_text)
            ende = int(ende_text)
            if start > ende:
                raise ValueError(f"Start groesser als Ende: {abschnitt!r}")
            ergebnis.update(range(start, ende + 1))
        else:
            if not abschnitt.isdigit():
                raise ValueError(f"Ungueltiger Abschnitt: {abschnitt!r}")
            ergebnis.add(int(abschnitt))
    return sorted(ergebnis)
```

Zwei verschachtelte Konfigurationen zusammenführen.

DER HAKEN

Verschachtelte Teile müssen zusammengeführt, einfache Werte aber ersetzt werden. Und das Original darf dabei nicht verändert werden — ein Fehler, den man erst Wochen später bemerkt.

WO DAS VORKOMMT

Standardeinstellungen mit Benutzereinstellungen überschreiben, in fast jeder Anwendung.

SCHNITTSTELLE verschmelzen(links, rechts) · _kopie(wert)

ABNAHME `python -m unittest tests.test_tief_verschmelzen` – nur Exitcode 0 zählt als gelöst

UMFANG 38 Zeilen

```
"""tief_verschmelzen: verschachtelte Woerterbuecher zusammenfuehren"""
from __future__ import annotations

from typing import Any

def verschmelzen(links: dict[str, Any], rechts: dict[str, Any]) -> dict[str, Any]:
    """Fuehrt zwei verschachtelte Woerterbuecher zusammen.

    Sind beide Werte zu einem Schluessel selbst Woerterbuecher, wird rekursiv
    verschmolzen. Sonst gewinnt der Wert aus "rechts". Beide Eingaben bleiben
    unveraendert, das Ergebnis teilt keine veraenderlichen Teile mit ihnen.
    """
    ergebnis = {}
    for schluessel in links:
        if schluessel in ergebnis:
            continue
        links_wert = links[schluessel]
        if isinstance(links_wert, dict):
            ergebnis[schluessel] = {k: _kopie(v) for k, v in links_wert.items()}
        else:
            ergebnis[schluessel] = _kopie(links_wert)
    for schluessel in rechts:
        rechts_wert = rechts[schluessel]
        if schluessel in ergebnis and isinstance(ergebnis[schluessel], dict) \
            and isinstance(rechts_wert, dict):
            ergebnis[schluessel] = verschmelzen(ergebnis[schluessel], rechts_wert)
        elif isinstance(rechts_wert, dict):
            ergebnis[schluessel] = {k: _kopie(v) for k, v in rechts_wert.items()}
        else:
            ergebnis[schluessel] = _kopie(rechts_wert)
    return ergebnis

def _kopie(wert: Any) -> Any:
    """Erstellt einen ungeteilten Abdruck von "wert".

    Woerterbuecher werden tief kopiert, damit das Ergebnis keine
    veraenderlichen Teile mit den Eingaben teilt. Alle anderen Werte
    bleiben unveraendert (die Aufgabe kennt nur Woerterbuecher und
    einfache Werte).
    """
    if isinstance(wert, dict):
        return {k: _kopie(v) for k, v in wert.items()}
    return wert
```

Eine verschachtelte Struktur zu einer flachen Liste mit Punktpfaden machen.

DER HAKEN

Listen und Wörterbücher verhalten sich unterschiedlich, und die Verschachtelung kann beliebig tief sein — Rekursion mit Sonderfällen.

WO DAS VORKOMMT

Formulardaten, Suchindizes, Protokollformate.

SCHNITTSTELLE `flach(daten, trenner)`

ABNAHME `python -m unittest tests.test_flach_klopfen` – nur Exitcode 0 zählt als gelöst

UMFANG 18 Zeilen

```
"""flach_klopfen: verschachteltes Woerterbuch flach machen"""
from __future__ import annotations

from typing import Any

def flach(daten: dict[str, Any], trenner: str = ".") -> dict[str, Any]:
    """Macht ein verschachteltes Woerterbuch flach.

    {"a": {"b": 1}} wird zu {"a.b": 1}. Ein leeres Unterwoerterbuch bleibt als
    Blatt erhalten: {"a": {}} wird zu {"a": {}}. Der Trenner ist einstellbar.
    """
    ergebnis: dict[str, Any] = {}
    for schlüssel, wert in daten.items():
        if isinstance(wert, dict) and wert:
            for unter_schlüssel, unter_wert in flach(wert, trenner=trenner).items():
                ergebnis[
                    f"{schlüssel}{trenner}{unter_schlüssel}"
                ] = unter_wert
        else:
            ergebnis[schlüssel] = wert
    return ergebnis
```

Aus einer verschachtelten Struktur einen Wert über a.b.c herausholen.

DER HAKEN

Fehlende Zwischenstufen dürfen nicht abstürzen, und ein Punkt im Schlüsselnamen selbst macht die Sache mehrdeutig.

WO DAS VORKOMMT

Konfigurationszugriff, Vorlagensprachen, API-Antworten.

SCHNITTSTELLE `holen(daten, pfad)`

ABNAHME `python -m unittest tests.test_punktpfad` – nur Exitcode 0 zählt als gelöst

UMFANG 27 Zeilen

```
"""punktpfad: Wert ueber einen Punktpfad aus einer Struktur holen"""
from __future__ import annotations

from typing import Any

def holen(daten: Any, pfad: str) -> Any:
    """Holt einen Wert ueber einen Pfad wie "a.b.0".

    Abschnitte gehen durch Woerterbuecher; ein rein numerischer Abschnitt
    indiziert eine Liste. Ein fehlender Schluessel wirft KeyError, ein
    ungueltiger Listenindex IndexError, ein leerer Pfad ValueError. Fuehrt der
    Pfad in einen einfachen Wert hinein, wird TypeError geworfen.
    """
    if not pfad:
        raise ValueError("leerer Pfad")

    wert = daten
    for abschnitt in pfad.split("."):
        if isinstance(wert, list):
            try:
                index = int(abschnitt)
            except ValueError:
                raise IndexError(
                    f"{abschnitt!r} ist kein gueltiger Listenindex"
                ) from None
            wert = wert[index]
        elif isinstance(wert, dict):
            wert = wert[abschnitt]
        else:
            raise TypeError("Pfad fuehrt in einen einfachen Wert hinein")
    return wert
```

Der Rückweg: aus 1,2,3,5 wieder 1-3,5 machen.

DER HAKEN

Zwei aufeinanderfolgende Zahlen werden üblicherweise nicht zu einem Bereich zusammengezogen — 1,2 bleibt 1,2 und wird nicht zu 1-2.

WO DAS VORKOMMT

Dieselben Stellen, nur in der Ausgaberichtung.

SCHNITTSTELLE `kompakt(zahlen) · _lauf(start, ende)`

ABNAHME `python -m unittest tests.test_bereich_kompakt` — nur Exitcode 0 zählt als gelöst

UMFANG 27 Zeilen

```
"""bereich_kompakt: Zahlenliste zu Bereichen zusammenfassen"""
from __future__ import annotations

def kompakt(zahlen: list[int]) -> str:
    """Fasst eine Zahlenliste zu einer Bereichsangabe zusammen.

    [1, 2, 3, 5, 7, 8, 9] wird zu "1-3,5,7-9". Ein zusammenhaengender Lauf ab
    zwei Zahlen wird zu "von-bis", eine einzelne Zahl bleibt stehen. Die Eingabe
    wird vorher sortiert und entdoppelt. Leere Eingabe ergibt "".
    """
    werte = sorted(set(zahlen))
    if not werte:
        return ""
    stuecke: list[str] = []
    lauf_start = werte[0]
    lauf_ende = werte[0]
    for wert in werte[1:]:
        if wert == lauf_ende + 1:
            lauf_ende = wert
        else:
            stuecke.append(_lauf(lauf_start, lauf_ende))
            lauf_start = lauf_ende = wert
    stuecke.append(_lauf(lauf_start, lauf_ende))
    return ",".join(stuecke)

def _lauf(start: int, ende: int) -> str:
    """Ein zusammenhaengender Lauf: ab zwei Zahlen als "von-bis"."""
    if start == ende:
        return str(start)
    return f"{start}-{ende}"
```

Rechnen in umgekehrter polnischer Notation: 3 4 + statt 3 + 4.

DER HAKEN

Ein Stapel, der leerlaufen kann. Zu wenige Zahlen für einen Operator, oder am Ende mehr als ein Ergebnis — beides ist ein Fehler in der Eingabe.

WO DAS VORKOMMT

Taschenrechner von HP, PostScript, und der Zwischenschritt fast jedes Übersetzers.

SCHNITTSTELLE `auswerten(ausdruck)`

ABNAHME `python -m unittest tests.test_rpn_rechner` – nur Exitcode 0 zählt als gelöst

UMFANG 42 Zeilen

```
"""rpn_rechner: Ausdruck in umgekehrter polnischer Notation auswerten"""
from __future__ import annotations

def auswerten(ausdruck: str) -> float:
    """Wertet einen Ausdruck in umgekehrter polnischer Notation aus.

    "3 4 +\" ergibt 7.0. Operatoren sind +, -, * und /. Getrennt wird durch
    beliebigen Leerraum. Das Ergebnis ist immer ein float. Division durch Null
    wirft ZeroDivisionError. Ein leerer Ausdruck, ein unbekanntes Zeichen, zu
    wenige Werte fuer einen Operator oder mehr als ein Rest auf dem Stapel
    werfen ValueError.
    """
    tokens = ausdruck.split()
    if not tokens:
        raise ValueError("Leerer Ausdruck.")

    stapel: list[float] = []
    for token in tokens:
        if token in {"+", "-", "*", "/"}:
            if len(stapel) < 2:
                raise ValueError(
                    f"Zu wenige Werte fuer Operator '{token}'."
                )
            rechts = stapel.pop()
            links = stapel.pop()
            if token == "+":
                stapel.append(links + rechts)
            elif token == "-":
                stapel.append(links - rechts)
            elif token == "*":
                stapel.append(links * rechts)
            else: # "/"
                if rechts == 0:
                    raise ZeroDivisionError("Division durch Null.")
                stapel.append(links / rechts)
        else:
            try:
                stapel.append(float(token))
            except ValueError:
                raise ValueError(f"Ungueltiger Token: {token!r}.") from None

    if len(stapel) != 1:
        raise ValueError(
            f"Ungueltiger Ausdruck: {len(stapel)} Werte bleiben auf dem Stapel."
        )
    return stapel[0]
```


Rechnen, wie ein Mensch schreibt: mit Klammern und Punkt vor Strich.

DER HAKEN

Vorrangregeln und Klammerung zugleich. Die anspruchsvollste Aufgabe der ganzen Sammlung — und diejenige, an der ein früherer Lauf sein gesamtes Zeitbudget verbrannt hat.

WO DAS VORKOMMT

Tabellenkalkulationen, Formelfelder, jede Programmiersprache.

SCHNITTSTELLE `_tokenisieren(ausdruck) · _nachrichten(token) ·
_berechnen(rpn) · auswerten(ausdruck)`

ABNAHME `python -m unittest tests.test_infix_rechner` – nur Exitcode 0
zählt als gelöst

UMFANG 95 Zeilen

```

"""infix_rechner: Klammerausdruck mit Punkt vor Strich auswerten"""
from __future__ import annotations

_VORRANG = {"+": 1, "-": 1, "*": 2, "/": 2}

def _tokenisieren(ausdruck: str) -> list[str]:
    """Zerlegt den Ausdruck in Zahlen, Operatoren und Klammern."""
    token: list[str] = []
    i = 0
    laenge = len(ausdruck)
    while i < laenge:
        c = ausdruck[i]
        if c.isspace():
            i += 1
            continue
        if c in "+-*/()":
            token.append(c)
            i += 1
            continue
        if c.isdigit() or c == ".":
            j = i
            punkte = 0
            while j < laenge and (ausdruck[j].isdigit() or ausdruck[j] == "."):
                if ausdruck[j] == ".":
                    punkte += 1
                j += 1
            zahl = ausdruck[i:j]
            if punkte > 1:
                raise ValueError(f"unzulässige Zahl: {zahl!r}")
            token.append(zahl)
            i = j
            continue
        raise ValueError(f"unbekanntes Zeichen: {c!r}")
    return token

def _nachrichten(token: list[str]) -> list[tuple[str, float | None]]:
    """Wandelt Infix in umgekehrte polnische Notation um (Shunting-Yard)."""
    stapel: list[str] = []
    ausgabe: list[tuple[str, float | None]] = []
    for t in token:
        if t[0].isdigit() or (t[0] == "." and len(t) > 1):
            ausgabe.append((t, float(t)))
        elif t in _VORRANG:
            while stapel and stapel[-1] in _VORRANG and _VORRANG[stapel[-1]] >= _VORRANG[t]:
                ausgabe.append((stapel.pop(), None))
            stapel.append(t)
        elif t == "(":
            stapel.append(t)
        elif t == ")":
            found = False
            while stapel and stapel[-1] != "(":
                ausgabe.append((stapel.pop(), None))
            found = True
            if not stapel or stapel[-1] != "(":
                raise ValueError("unpassende Klammer")
            stapel.pop() # das '(' weg
    while stapel:
        op = stapel.pop()
        if op == "(":
            raise ValueError("unpassende Klammer")
        ausgabe.append((op, None))
    return ausgabe

def _berechnen(rpn: list[tuple[str, float | None]]) -> float:
    """Wertet den RPN-Baum mit einem Stapel aus."""
    stack: list[float] = []
    for token, wert in rpn:
        if wert is not None:
            stack.append(wert)
        else:
            if len(stack) < 2:
                raise ValueError("fehlender Operand")
            b = stack.pop()

```

```

        a = stack.pop()
        if token == "+":
            stack.append(a + b)
        elif token == "-":
            stack.append(a - b)
        elif token == "*":
            stack.append(a * b)
        else: # "/"
            if b == 0:
                raise ZeroDivisionError("Division durch Null")
            stack.append(a / b)
    if len(stack) != 1:
        raise ValueError("fehlender Operand")
    return stack[0]

```

```

def auswerten(ausdruck: str) -> float:
    """Wertet einen Ausdruck mit + - * / und Klammern aus.

```

Punkt vor Strich gilt, Klammern haben Vorrang. Zahlen sind nicht negativ und dürfen einen Dezimalpunkt enthalten; Leerraum wird ignoriert. Das Ergebnis ist immer ein float. Ein leerer Ausdruck, ein unbekanntes Zeichen, eine unpassende Klammer oder ein fehlender Operand werfen ValueError. Division durch Null wirft ZeroDivisionError.

```

    """
    token = _tokenisieren(ausdruck)
    if not token:
        raise ValueError("leerer Ausdruck")
    return _berechnen(_nachrichten(token))

```

Die Prüfziffer einer zehnstelligen Buchnummer.

DER HAKEN

Die Prüfziffer kann ein X sein — sie steht für zehn. Wer nur Ziffern zulässt, weist gültige Bücher ab.

WO DAS VORKOMMT

Bibliothekskataloge, Buchhandel, Antiquariate.

SCHNITTSTELLE `pruefziffer(neun) · ist_gueltig(isbn)`

ABNAHME `python -m unittest tests.test_isbn10` – nur Exitcode 0 zählt als gelöst

UMFANG 30 Zeilen

```
"""isbn10: ISBN-10 pruefen und Pruefziffer berechnen"""
from __future__ import annotations

def pruefziffer(neun: str) -> str:
    """Berechnet die zehnte Stelle zu neun Ziffern.

    Gewichte sind 10 bis 2, die Pruefziffer ergaenzt auf ein Vielfaches von 11.
    Der Wert 10 wird als "X" zurueckgegeben. Eine Eingabe, die nicht aus genau
    neun Ziffern besteht, wirft ValueError.
    """
    if len(neun) != 9 or not neun.isdigit():
        raise ValueError("Pruefziffer braucht genau neun Ziffern")
    total = sum(int(z) * (10 - i) for i, z in enumerate(neun))
    rest = (11 - total % 11) % 11
    return "X" if rest == 10 else str(rest)

def ist_gueltig(isbn: str) -> bool:
    """Prueft eine vollstaendige ISBN-10.

    Bindestriche und Leerzeichen werden ignoriert. Nur die letzte Stelle darf
    "X" oder "x" sein. Eine falsche Laenge oder ein ungueltiges Zeichen ergibt
    False statt einer Ausnahme.
    """
    zahlen = [z for z in isbn if z not in "- "]
    if len(zahlen) != 10:
        return False
    if not all(z.isdigit() for z in zahlen[:9]):
        return False
    letzte = zahlen[9]
    if letzte not in "0123456789Xx":
        return False
    wert = 10 if letzte in "Xx" else int(letzte)
    total = sum(int(z) * (10 - i) for i, z in enumerate(zahlen[:9])) + wert
    return total % 11 == 0
```

Die Prüfziffer des Strichcodes auf jeder Verpackung.

DER HAKEN

Abwechselnd mal eins und mal drei gewichten. Ob man mit der Eins oder der Drei beginnt, entscheidet über richtig und falsch.

WO DAS VORKOMMT

Jede Supermarktkasse, jedes Lagerverwaltungssystem.

SCHNITTSTELLE `pruefziffer(zwoelf) · ist_gueltig(code)`

ABNAHME `python -m unittest tests.test_ean13` – nur Exitcode 0 zählt als gelöst

UMFANG 19 Zeilen

```
"""ean13: EAN-13 Prüfziffer berechnen und prüfen"""
from __future__ import annotations

def pruefziffer(zwoelf: str) -> int:
    """Berechnet die dreizehnte Stelle zu zwoelf Ziffern.

    Von links werden die Stellen abwechselnd mit 1 und 3 gewichtet, beginnend
    mit 1. Die Prüfziffer ergaenzt die Summe auf ein Vielfaches von 10. Eine
    Eingabe, die nicht aus genau zwoelf Ziffern besteht, wirft ValueError.
    """
    if len(zwoelf) != 12 or not zwoelf.isdigit():
        raise ValueError("genau zwoelf Ziffern erwartet")
    total = sum(int(z) * (1 if i % 2 == 0 else 3) for i, z in enumerate(zwoelf))
    return (10 - total % 10) % 10

def ist_gueltig(code: str) -> bool:
    """Prueft einen vollstaendigen EAN-13-Code.

    Falsche Laenge oder ein Zeichen, das keine Ziffer ist, ergibt False.
    """
    if len(code) != 13 or not code.isdigit():
        return False
    return pruefziffer(code[:12]) == int(code[12])
```

Farbwerte zwischen #ff8800 und Rot/Grün/Blau umrechnen.

DER HAKEN

Die Kurzschreibweise. #f80 bedeutet dasselbe wie #ff8800, und beide Richtungen müssen zusammenpassen.

WO DAS VORKOMMT

Jede Website, jedes Grafikprogramm.

SCHNITTSTELLE `hex_zu_rgb(farbe) · rgb_zu_hex(rgb)`

ABNAHME `python -m unittest tests.test_hex_rgb` – nur Exitcode 0 zählt als gelöst

UMFANG 34 Zeilen

```
"""hex_rgb: Farbwerte zwischen Hex und RGB wandeln"""
from __future__ import annotations

def hex_zu_rgb(farbe: str) -> tuple[int, int, int]:
    """Wandelt "#aabbcc" oder die Kurzform "#abc" in ein RGB-Tripel.

    Die Raute ist optional, Gross- und Kleinschreibung egal. In der Kurzform
    wird jede Stelle verdoppelt: "#abc" entspricht "#aabbcc". Eine falsche
    Laenge oder eine ungueltige Hexziffer wirft ValueError.
    """
    if not isinstance(farbe, str):
        raise ValueError("farbe muss eine Zeichenkette sein")
    text = farbe.lstrip("#")
    if len(text) == 3:
        text = "".join(zeichen * 2 for zeichen in text)
    if len(text) != 6:
        raise ValueError(f"ungueltige Laenge: {farbe!r}")
    try:
        r = int(text[0:2], 16)
        g = int(text[2:4], 16)
        b = int(text[4:6], 16)
    except ValueError:
        raise ValueError(f"ungueltige Hexziffern: {farbe!r}") from None
    return (r, g, b)

def rgb_zu_hex(rgb: tuple[int, int, int]) -> str:
    """Wandelt ein RGB-Tripel in "#rrggbb", klein geschrieben.

    Ein Wert ausserhalb 0..255 wirft ValueError.
    """
    if len(rgb) != 3:
        raise ValueError("rgb muss genau drei Werte enthalten")
    out = []
    for wert in rgb:
        if not isinstance(wert, int) or isinstance(wert, bool) or not 0 <= wert <= 255:
            raise ValueError(f"Wert ausserhalb 0..255: {wert!r}")
        out.append(f"{wert:02x}")
    return "#" + "".join(out)
```

Die längste gemeinsame Teilfolge zweier Zeichenketten.

DER HAKEN

Teilfolge ist nicht Teilstring — die Zeichen müssen in der Reihenfolge stehen, aber nicht nebeneinander. Wieder eine Tabelle, wieder mit Rändern.

WO DAS VORKOMMT

Der Kern jedes Vergleichs zweier Dateiversionen.

SCHNITTSTELLE `laenge(links, rechts) · teilfolge(links, rechts)`

ABNAHME `python -m unittest tests.test_lcs_laenge` – nur Exitcode 0
zählt als gelöst

UMFANG 52 Zeilen

```

"""lcs_laenge: laengste gemeinsame Teilfolge zweier Zeichenketten"""
from __future__ import annotations

def laenge(links: str, rechts: str) -> int:
    """Laenge der laengsten gemeinsamen Teilfolge.

    Die Zeichen muessen in Reihenfolge vorkommen, aber nicht zusammenhaengend.
    Muss auch bei 200 Zeichen je Seite zuegig laufen, rohes Durchprobieren
    genuegt nicht.
    """
    n, m = len(links), len(rechts)
    if n == 0 or m == 0:
        return 0
    # Klassisches DP: t[i][j] = LCS-Laenge von links[:i] und rechts[:j]
    t = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        row_prev = t[i - 1]
        row_cur = t[i]
        for j in range(1, m + 1):
            if links[i - 1] == rechts[j - 1]:
                row_cur[j] = row_prev[j - 1] + 1
            else:
                a, b = row_prev[j], row_cur[j - 1]
                row_cur[j] = a if a >= b else b
    return t[n][m]

def teilfolge(links: str, rechts: str) -> str:
    """Eine laengste gemeinsame Teilfolge.

    Bei mehreren gleich langen Loesungen wird die zurueckgegeben, die beim
    Rueckverfolgen von hinten entsteht.
    """
    n, m = len(links), len(rechts)
    if n == 0 or m == 0:
        return ""
    t = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        row_prev = t[i - 1]
        row_cur = t[i]
        for j in range(1, m + 1):
            if links[i - 1] == rechts[j - 1]:
                row_cur[j] = row_prev[j - 1] + 1
            else:
                a, b = row_prev[j], row_cur[j - 1]
                row_cur[j] = a if a >= b else b

    # Rueckverfolgung von hinten; bei Gleichstand nach oben (links-Bevorzugung)
    result: list[str] = []
    i, j = n, m
    while i > 0 and j > 0:
        if links[i - 1] == rechts[j - 1]:
            result.append(links[i - 1])
            i -= 1
            j -= 1
        elif t[i - 1][j] >= t[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(result))

```


Zwei Zahlentabellen miteinander multiplizieren.

DER HAKEN

Die Dimensionen müssen zusammenpassen, und die Reihenfolge ist nicht vertauschbar. Drei ineinander verschachtelte Schleifen, in denen man leicht Zeile und Spalte verwechselt.

WO DAS VORKOMMT

Computergrafik, neuronale Netze, Statik.

SCHNITTSTELLE `multiplizieren(links, rechts)`

ABNAHME `python -m unittest tests.test_matrix_multiplizieren` – nur Exitcode 0 zählt als gelöst

UMFANG 31 Zeilen

```
"""matrix_multiplizieren: Produkt zweier Matrizen"""
from __future__ import annotations

def multiplizieren(links: list[list[float]], rechts: list[list[float]]) -> list[list[float]]:
    """Matrixprodukt links mal rechts.

    Die Spaltenzahl von links muss der Zeilenzahl von rechts entsprechen.
    Leere Matrizen, Zeilen unterschiedlicher Laenge oder unpassende Groessen
    werfen ValueError.
    """
    if not links or not rechts:
        raise ValueError("Matrizen duerfen nicht leer sein.")

    zeilen_l, spalten_l = len(links), len(links[0])
    zeilen_r, spalten_r = len(rechts), len(rechts[0])

    for zeile in links:
        if len(zeile) != spalten_l:
            raise ValueError("Unregelmaessige Matrix: Zeilen unterschiedlicher Laenge.")
    for zeile in rechts:
        if len(zeile) != spalten_r:
            raise ValueError("Unregelmaessige Matrix: Zeilen unterschiedlicher Laenge.")

    if spalten_l != zeilen_r:
        raise ValueError(
            "Unverkraemmelbar: Spaltenzahl der linken Matrix muss der "
            f"Zeilenzahl der rechten entsprechen ({spalten_l} != {zeilen_r})."
        )

    ergebnis: list[list[float]] = []
    for i in range(zeilen_l):
        zeile = [
            sum(links[i][k] * rechts[k][j] for k in range(zeilen_r))
            for j in range(spalten_r)
        ]
        ergebnis.append(zeile)
    return ergebnis
```

Eine Tabelle spiralförmig von aussen nach innen auslesen.

DER HAKEN

Vier Richtungen, vier Grenzen, und ein Abbruch, der weder eine Reihe doppelt liest noch eine auslässt. Rechteckige Tabellen sind der Fall, an dem die meisten Lösungen scheitern.

WO DAS VORKOMMT

Bildabtastung, Speicheranordnung, Programmerrätsel.

SCHNITTSTELLE `spirale(matrix)`

ABNAHME `python -m unittest tests.test_spirale` – nur Exitcode 0 zählt als gelöst

UMFANG 34 Zeilen

```
"""spirale: Matrix spiralförmig auslesen"""
from __future__ import annotations

def spirale(matrix: list[list[int]]) -> list[int]:
    """Liest eine Matrix im Uhrzeigersinn von aussen nach innen aus.

    Begonnen wird links oben nach rechts. Die Matrix muss nicht quadratisch
    sein. Eine leere Matrix ergibt eine leere Liste. Unterschiedlich lange
    Zeilen werfen ValueError.
    """
    if not matrix:
        return []
    breite = len(matrix[0])
    if any(len(zeile) != breite for zeile in matrix):
        raise ValueError("Zeilen haben unterschiedliche Laengen.")
    if breite == 0:
        return []

    oben, unten = 0, len(matrix) - 1
    links, rechts = 0, breite - 1
    ergebnis: list[int] = []

    while oben <= unten and links <= rechts:
        for spalte in range(links, rechts + 1):
            ergebnis.append(matrix[oben][spalte])
        oben += 1
        for zeile in range(oben, unten + 1):
            ergebnis.append(matrix[zeile][rechts])
        rechts -= 1
        if oben <= unten:
            for spalte in range(rechts, links - 1, -1):
                ergebnis.append(matrix[unten][spalte])
            unten -= 1
        if links <= rechts:
            for zeile in range(unten, oben - 1, -1):
                ergebnis.append(matrix[zeile][links])
            links += 1

    return ergebnis
```

Eine Tabelle um 90 Grad drehen.

DER HAKEN

Die Drehrichtung. Transponieren und Spiegeln in der falschen Reihenfolge dreht in die andere Richtung, und das fällt nur bei unsymmetrischen Beispielen auf.

WO DAS VORKOMMT

Bildbearbeitung, Tetris, Kartenanzeigen.

SCHNITTSTELLE `_pruefen(matrix) · im_uhrzeigersinn(matrix) · gegen_uhrzeigersinn(matrix)`

ABNAHME `python -m unittest tests.test_matrix_drehen` – nur Exitcode 0 zählt als gelöst

UMFANG 30 Zeilen

```
"""matrix_drehen: Matrix um 90 Grad drehen"""
from __future__ import annotations

def _pruefen(matrix: list[list[int]]) -> None:
    """Werft ValueError, wenn die Zeilen unterschiedlich lang sind."""
    breite = len(matrix[0]) if matrix else 0
    for zeile in matrix:
        if len(zeile) != breite:
            raise ValueError("Zeilen haben unterschiedliche Laenge.")

def im_uhrzeigersinn(matrix: list[list[int]]) -> list[list[int]]:
    """Dreht die Matrix um 90 Grad im Uhrzeigersinn.

    Aus einer m mal n Matrix wird eine n mal m Matrix. Die Eingabe bleibt
    unverändert. Unterschiedlich lange Zeilen werfen ValueError.
    """
    if not matrix:
        return []
    _pruefen(matrix)
    zeilen = len(matrix)
    spalten = len(matrix[0])
    return [[matrix[zeilen - 1 - i][j] for i in range(zeilen)] for j in range(spalten)]

def gegen_uhrzeigersinn(matrix: list[list[int]]) -> list[list[int]]:
    """Dreht die Matrix um 90 Grad gegen den Uhrzeigersinn.

    Aus einer m mal n Matrix wird eine n mal m Matrix. Die Eingabe bleibt
    unverändert. Unterschiedlich lange Zeilen werfen ValueError.
    """
    if not matrix:
        return []
    _pruefen(matrix)
    zeilen = len(matrix)
    spalten = len(matrix[0])
    return [[matrix[j][spalten - 1 - i] for j in range(zeilen)] for i in range(spalten)]
```

Die Fläche eines beliebigen Vielecks aus seinen Eckpunkten.

DER HAKEN

Der Betrag am Ende. Ohne ihn kommt bei umgekehrtem Umlaufsinn eine negative Fläche heraus.

WO DAS VORKOMMT

Landvermessung, Geoinformationssysteme, Bauplanung.

SCHNITTSTELLE `flaeche(punkte)`

ABNAHME `python -m unittest tests.test_polygon_flaeche` – nur Exitcode 0 zählt als gelöst

UMFANG 16 Zeilen

```
"""polygon_flaeche: Fläche eines Polygons nach der Trapezformel"""
from __future__ import annotations

def flaeche(punkte: list[tuple[float, float]]) -> float:
    """Fläche eines einfachen Polygons.

    Die Punkte sind Paare (x, y) in Reihenfolge des Umlaufs; der letzte Punkt
    wird automatisch mit dem ersten verbunden. Das Ergebnis ist immer nicht
    negativ, unabhaengig vom Umlaufsinn. Weniger als drei Punkte werfen
    ValueError.
    """
    if len(punkte) < 3:
        raise ValueError("Ein Polygon braucht mindestens drei Punkte.")
    summe = 0.0
    for i, (x0, y0) in enumerate(punkte):
        x1, y1 = punkte[(i + 1) % len(punkte)]
        summe += x0 * y1 - x1 * y0
    return abs(summe) / 2.0
```

Abhängigkeiten in eine Reihenfolge bringen — was muss vor was erledigt sein.

DER HAKEN

Ein Zyklus muss erkannt werden, und bei mehreren gültigen Reihenfolgen soll die alphabetisch kleinste herauskommen. Genau hier hat Ben eine Endlosschleife geschrieben: er lief über eine Liste, in die er gleichzeitig einfügte.

WO DAS VORKOMMT

Paketverwaltungen, Bauwerkzeuge, Projektpläne.

SCHNITTSTELLE `sortieren(graph)`

ABNAHME `python -m unittest tests.test_topologisch` – nur Exitcode 0 zählt als gelöst

UMFANG 33 Zeilen

```
"""topologisch: Abhängigkeiten in eine Reihenfolge bringen"""
from __future__ import annotations

def sortieren(graph: dict[str, list[str]]) -> list[str]:
    """Topologische Sortierung eines gerichteten Graphen.

    Der Graph bildet jeden Knoten auf seine Nachfolger ab; ein Knoten, der nur
    als Nachfolger vorkommt, gehoert trotzdem dazu. Sind mehrere Reihenfolgen
    moeglich, wird die alphabetisch kleinste geliefert. Ein Zyklus wirft
    ValueError.
    """
    nachfolger: dict[str, set[str]] = {knoten: set() for knoten in graph}
    for knoten, zielliste in graph.items():
        nachfolger.setdefault(knoten, set()).update(zielliste)
        for ziel in zielliste:
            nachfolger.setdefault(ziel, set())

    einzug: dict[str, int] = {k: 0 for k in nachfolger}
    for knoten, ziel in nachfolger.items():
        for z in ziel:
            einzug[z] += 1

    bereit = sorted(k for k, v in einzug.items() if v == 0)
    ordnung: list[str] = []
    while bereit:
        knoten = bereit.pop(0)
        ordnung.append(knoten)
        neu: list[str] = []
        for z in nachfolger[knoten]:
            einzug[z] -= 1
            if einzug[z] == 0:
                neu.append(z)
        if neu:
            bereit = sorted(bereit + neu)
    if len(ordnung) != len(nachfolger):
        raise ValueError("Zyklus im Graphen")
    return ordnung
```

Den kürzesten Weg in einem Netz ohne Gewichte finden.

DER HAKEN

Eine Warteschlange, kein Stapel. Wer den Stapel nimmt, findet zwar einen Weg, aber nicht den kürzesten — und der Unterschied fällt bei kleinen Beispielen nicht auf.

WO DAS VORKOMMT

Routenplanung, soziale Netzwerke, Spielelogik.

SCHNITTSTELLE `kuerzester_weg(graph, von, bis)`

ABNAHME `python -m unittest tests.test_graph_bfs` — nur Exitcode 0 zählt als gelöst

UMFANG 42 Zeilen

```

"""graph_bfs: kuerzester Weg in einem ungewichteten Graphen"""
from __future__ import annotations

from collections import deque

def kuerzester_weg(graph: dict[str, list[str]], von: str, bis: str) -> list[str]:
    """Kuerzester Weg als Liste der Knoten, Start und Ziel eingeschlossen.

    Der Graph wird als ungerichtet behandelt: steht b in der Nachbarliste von a,
    gilt die Kante auch umgekehrt. Bei mehreren gleich kurzen Wegen wird der
    alphabetisch kleinste geliefert. Sind Start und Ziel gleich, ist der Weg
    einelementig. Ein unbekannter Knoten wirft KeyError, ein fehlender Weg
    ValueError.
    """
    # Alle bekannten Knoten: Keys der Abbildung zugaendlich jede Nachbarliste.
    # Im Setup-Graphen stehen viele Knoten (z. B. "e") nur in Nachbarlisten,
    # nicht selbst als Key – beides sind echte Knoten.
    knoten = set(graph)
    for liste in graph.values():
        knoten.update(liste)
    if von not in knoten:
        raise KeyError(von)
    if bis not in knoten:
        raise KeyError(bis)

    # Ungereichtes Nachbarn: Kanten beider Richtungen, sortiert damit bei
    # gleicher Weglaenge der alphabetisch kleinste Weg gewinnt.
    nachbarn: dict[str, list[str]] = {knoten: [] for knoten in knoten}
    for knoten, liste in graph.items():
        for nachbar in liste:
            nachbarn[knoten].append(nachbar)
            if nachbar in nachbarn:
                nachbarn[nachbar].append(knoten)
    for knoten in nachbarn:
        nachbarn[knoten].sort()

    voraus: dict[str, str | None] = {von: None}
    warte = deque([von])
    while warte:
        aktueller = warte.popleft()
        if aktueller == bis:
            weg: list[str] = []
            k: str | None = aktueller
            while k is not None:
                weg.append(k)
                k = voraus[k]
            return weg[::-1]
        for nachbar in nachbarn[aktueller]:
            if nachbar not in voraus:
                voraus[nachbar] = aktueller
                warte.append(nachbar)

    raise ValueError(f"Kein Weg von {von!r} nach {bis!r}")

```

Zusammenhängende Gruppen bestimmen: wer gehört mit wem zusammen.

DER HAKEN

Ohne die beiden üblichen Kniffe wird die Struktur bei vielen Verbindungen sehr langsam. Mit ihnen ist sie fast so schnell, wie es überhaupt geht.

WO DAS VORKOMMT

Bildsegmentierung, Netzwerkanalyse, Freundeskreise.

SCHNITTSTELLE `gruppen(knoten, kanten)`

ABNAHME `python -m unittest tests.test_union_find` – nur Exitcode 0 zählt als gelöst

UMFANG 29 Zeilen

```
"""union_find: zusammenhängende Gruppen bestimmen"""
from __future__ import annotations

def gruppen(knoten: list[str], kanten: list[tuple[str, str]]) -> list[list[str]]:
    """Zusammenhangskomponenten eines ungerichteten Graphen.

    Jede Gruppe ist aufsteigend sortiert, die Gruppen sind nach ihrem kleinsten
    Element sortiert. Ein einzelner Knoten ohne Kante bildet eine eigene Gruppe.
    Eine Kante mit einem Knoten, der nicht in der Knotenliste steht, wirft
    KeyError.
    """
    eltern = {k: k for k in knoten}

    def find(x):
        while eltern[x] != x:
            eltern[x] = eltern[eltern[x]]
            x = eltern[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra != rb:
            eltern[rb] = ra

    for a, b in kanten:
        if a not in eltern:
            raise KeyError(a)
        if b not in eltern:
            raise KeyError(b)
        union(a, b)

    sammel: dict[str, list[str]] = {}
    for k in knoten:
        sammel.setdefault(find(k), []).append(k)

    return [sorted(mitglieder) for mitglieder in sorted(sammel.values())]
```


Alle möglichen Teilmengen einer Liste aufzählen.

DER HAKEN

Die Anzahl verdoppelt sich mit jedem Element. Und die leere Menge gehört dazu
— die wird gern vergessen.

WO DAS VORKOMMT

Kombinatorik, Testabdeckung, Konfigurationsvarianten.

SCHNITTSTELLE `potenzmenge(werte)`

ABNAHME `python -m unittest tests.test_potenzmenge` – nur Exitcode 0
zählt als gelöst

UMFANG 20 Zeilen

```
"""potenzmenge: alle Teilmengen einer Liste"""
from __future__ import annotations

from itertools import combinations

def potenzmenge(werte: list[int]) -> list[list[int]]:
    """Alle Teilmengen, sortiert nach Groesse.

    Innerhalb gleicher Groesse gilt die Reihenfolge der Eingabe. Die leere
    Teilmenge kommt zuerst. Dubletten in der Eingabe werden vorher entfernt,
    wobei das erste Vorkommen die Position bestimmt.
    """
    einzig: list[int] = []
    gesehen: set[int] = set()
    for w in wert:
        if w not in gesehen:
            gesehen.add(w)
            einzig.append(w)

    ergebnis: list[list[int]] = []
    for groesse in range(len(einzig) + 1):
        for kombi in combinations(einzig, groesse):
            ergebnis.append(list(kombi))
    return ergebnis
```

Eine Zahlenkodierung, bei der sich von einem Schritt zum nächsten nur ein einziges Bit ändert.

DER HAKEN

Der Rückweg ist deutlich kniffliger als der Hinweg und lässt sich nicht durch Umkehren derselben Formel gewinnen.

WO DAS VORKOMMT

Drehgeber in Maschinen, Fehlervermeidung bei Analog-Digital-Wandlern.

SCHNITTSTELLE `_pruefen(zahl) · zu_gray(zahl) · von_gray(gray)`

ABNAHME `python -m unittest tests.test_graycode` – nur Exitcode 0 zählt als gelöst

UMFANG 17 Zeilen

```
"""graycode: Gray-Code hin und zurueck wandeln"""
from __future__ import annotations

def _pruefen(zahl: int) -> None:
    if zahl < 0:
        raise ValueError("nur nicht negative Zahlen erlaubt")

def zu_gray(zahl: int) -> int:
    """Wandelt eine nicht negative Zahl in ihren Gray-Code."""
    _pruefen(zahl)
    return zahl ^ (zahl >> 1)

def von_gray(gray: int) -> int:
    """Wandelt einen Gray-Code zurueck in die Zahl."""
    _pruefen(gray)
    zahl = 0
    while gray:
        zahl ^= gray
        gray >>= 1
    return zahl
```

Die Bits einer Zahl innerhalb einer festen Breite umdrehen.

DER HAKEN

Die feste Breite. Führende Nullen zählen mit, und ohne sie kommt eine ganz andere Zahl heraus.

WO DAS VORKOMMT

Signalverarbeitung, Hardwareansteuerung, Kompressionsverfahren.

SCHNITTSTELLE `umkehren(wert, breite) · zaehle_bits(wert)`

ABNAHME `python -m unittest tests.test_bit_drehen` – nur Exitcode 0 zählt als gelöst

UMFANG 23 Zeilen

```
"""bit_drehen: Bits innerhalb einer festen Breite umkehren"""
from __future__ import annotations

def umkehren(wert: int, breite: int) -> int:
    """Spiegelt die Bits von wert innerhalb der angegebenen Breite.

    umkehren(0b0001, 4) ergibt 0b1000. Breite kleiner 1, negativer Wert oder
    ein Wert, der nicht in breite Bits passt, wirft ValueError.
    """
    if breite < 1:
        raise ValueError("breite muss mindestens 1 sein")
    if wert < 0:
        raise ValueError("wert darf nicht negativ sein")
    if wert >= (1 << breite):
        raise ValueError("wert passt nicht in die angegebene Breite")
    umgedreht = 0
    for i in range(breite):
        if (wert >> i) & 1:
            umgedreht |= 1 << (breite - 1 - i)
    return umgedreht

def zaehle_bits(wert: int) -> int:
    """Anzahl der gesetzten Bits. Negativer Wert wirft ValueError."""
    if wert < 0:
        raise ValueError("wert darf nicht negativ sein")
    return bin(wert).count("1")
```

Und 308 weitere Aufgaben in 5 Abteilungen

68 einzeln geschriebene plus 308 hier ergibt **376** — die Zahl von ganz oben. Diese 308 wurden maschinell erzeugt: gleiche Form, andere Parameter. Eine andere Zahlenbasis, eine andere Fensterbreite, eine andere Schranke. Die Erklärung steht deshalb einmal je Abteilung, darunter aber **jede einzelne Aufgabe** mit ihrer Nummer und dem Code, den der Agent für genau sie geschrieben hat.

Die Ziffern einer Zahl addieren, und das so lange wiederholen, bis das Ergebnis unter einer vorgegebenen Schranke liegt.

DER HAKEN

Die Schranke wird je Variante ausgewürfelt. Liegt sie unter zehn, kann die Schleife nie enden — das ist beim Bau des Generators tatsächlich passiert und musste behoben werden.

Alle 62 einzeln. Hinter jeder Nummer steht ihr erster Testfall – daran sieht man, dass es verschiedene Aufgaben sind und nicht dieselbe 62 mal. Der Code jeder einzelnen Variante steht auf der Webseite; auf Papier würde er den Umfang vervielfachen, ohne etwas zu zeigen, was die Geschwister nicht auch zeigen.

069	wurzel_quersumme(63) → 9	54 s
074	wurzel_quersumme(781) → 7	59 s
079	wurzel_quersumme(2950) → 7	47 s
084	wurzel_quersumme(15842) → 2	54 s
089	wurzel_quersumme(929397) → 3	46 s
094	wurzel_quersumme(28) → 1	43 s
099	wurzel_quersumme(90) → 9	67 s
104	wurzel_quersumme(50) → 5	48 s
109	wurzel_quersumme(5) → 5	50 s
114	wurzel_quersumme(518884) → 7	49 s
119	wurzel_quersumme(52) → 7	51 s
124	wurzel_quersumme(390899) → 11	47 s
129	wurzel_quersumme(373815) → 9	53 s
134	wurzel_quersumme(3211) → 7	52 s
139	wurzel_quersumme(807939) → 9	40 s
144	wurzel_quersumme(8) → 8	108 s
149	wurzel_quersumme(9152) → 8	49 s
154	wurzel_quersumme(4) → 4	56 s
159	wurzel_quersumme(610) → 7	59 s
164	wurzel_quersumme(216579) → 3	47 s
169	wurzel_quersumme(74) → 11	49 s
174	wurzel_quersumme(8) → 8	51 s
179	wurzel_quersumme(852) → 6	47 s
184	wurzel_quersumme(1031) → 5	52 s
189	wurzel_quersumme(0) → 0	44 s
194	wurzel_quersumme(2) → 2	51 s
199	wurzel_quersumme(200848) → 4	52 s
204	wurzel_quersumme(67) → 4	65 s
209	wurzel_quersumme(4813) → 7	52 s
214	wurzel_quersumme(159) → 6	53 s
219	wurzel_quersumme(688178) → 11	56 s
224	wurzel_quersumme(35185) → 4	71 s
229	wurzel_quersumme(29510) → 8	50 s
234	wurzel_quersumme(69756) → 6	58 s

239	wurzel_quersumme(6663) → 3	50 s
244	wurzel_quersumme(5620) → 4	46 s
249	wurzel_quersumme(751) → 4	60 s
254	wurzel_quersumme(937) → 1	52 s
259	wurzel_quersumme(1) → 1	62 s
264	wurzel_quersumme(9491) → 5	46 s
269	wurzel_quersumme(530649) → 9	52 s
274	wurzel_quersumme(6) → 6	52 s
279	wurzel_quersumme(9) → 9	55 s
284	wurzel_quersumme(44592) → 6	55 s
289	wurzel_quersumme(500413) → 4	53 s
294	wurzel_quersumme(775) → 1	55 s
299	wurzel_quersumme(737) → 8	82 s
304	wurzel_quersumme(1182) → 3	54 s
309	wurzel_quersumme(865) → 10	49 s
314	wurzel_quersumme(30226) → 4	50 s
319	wurzel_quersumme(39873) → 3	51 s
324	wurzel_quersumme(42) → 6	48 s
329	wurzel_quersumme(997754) → 5	77 s
334	wurzel_quersumme(2180) → 11	47 s
339	wurzel_quersumme(1111) → 4	55 s
344	wurzel_quersumme(62912) → 2	105 s
349	wurzel_quersumme(723443) → 5	67 s
354	wurzel_quersumme(179) → 8	54 s
359	wurzel_quersumme(37052) → 8	62 s
364	wurzel_quersumme(987) → 6	49 s
369	wurzel_quersumme(75526) → 7	65 s
374	wurzel_quersumme(67409) → 8	48 s

Wiederholte Zeichen zusammenfassen, aus aaab also a^3b^1 .

DER HAKEN

Trennzeichen und Format wechseln je Variante. Die leere Eingabe muss eine leere Ausgabe ergeben und nicht abstürzen.

Alle 62 einzeln. Hinter jeder Nummer steht ihr erster Testfall – daran sieht man, dass es verschiedene Aufgaben sind und nicht dieselbe 62 mal. Der Code jeder einzelnen Variante steht auf der Webseite; auf Papier würde er den Umfang vervielfachen, ohne etwas zu zeigen, was die Geschwister nicht auch zeigen.

070	kodieren('babbbbaa') → 'b*1 a*1 b*3 a*2'	42 s
075	kodieren('abbaabaabbbcb') → 'a*1:b*2:a*2:b*1:a*2:b*3:a*1:b*1'	56 s
080	kodieren('') → ''	44 s
085	kodieren('aaca') → 'a*2-c*1-a*2'	46 s
090	kodieren('cccabbacbbb') → 'c*3:a*1:b*3:a*1:c*1:b*4'	47 s
095	kodieren('cbbbbcbaca') → 'c*1:b*5:c*1:a*1:c*1:a*1'	44 s
100	kodieren('aabcbabbbcb') → 'a*2-b*1-c*1-b*2-a*1-b*4-c*1-b*1'	54 s
105	kodieren('acbbababbbabc') → 'a*1-c*1-b*2-a*1-b*1-a*1-b*3-a*1-...	74 s
110	kodieren('') → ''	42 s
115	kodieren('bbbababcb') → 'b*3:a*1:b*1:a*1:b*1:c*1:b*1'	48 s
120	kodieren('abcbcacabab') → 'a*1 b*1 c*1 b*1 c*1 b*1 a*1 c*1 ...	43 s
125	kodieren('baaacbbcca') → 'b*1:a*3:c*1:b*2:c*2:a*1'	42 s
130	kodieren('bbabab') → 'b*2-a*1-b*1-a*1-b*1'	46 s
135	kodieren('abba') → 'a*1:b*2:a*1'	44 s
140	kodieren('babbbababbaac') → 'b*1:a*1:b*3:a*1:b*1:a*1:b*2:a*2:...	46 s
145	kodieren('bbcacabba') → 'b*2-c*1-a*1-c*1-a*1-b*2-a*1'	55 s
150	kodieren('acacabb') → 'a*1 c*1 a*1 c*1 a*1 b*2'	47 s
155	kodieren('aab') → 'a*2 b*1'	44 s
160	kodieren('bbbbbbca') → 'b*6:c*1:a*1'	42 s
165	kodieren('aaaabcaacbcc') → 'a*4:b*1:c*1:a*2:c*1:b*1:c*2'	47 s
170	kodieren('abacbaa') → 'a*1:b*1:a*1:c*1:b*1:a*2'	44 s
175	kodieren('bba') → 'b*2-a*1'	46 s
180	kodieren('cbabbc') → 'c*1:b*1:a*1:b*2:c*1'	44 s
185	kodieren('ababbbab') → 'a*1-b*1-a*1-b*3-a*1-b*1'	44 s
190	kodieren('baaabbbbbb') → 'b*1:a*3:b*7'	46 s
195	kodieren('babbacac') → 'b*1:a*1:b*2:a*1:c*1:a*1:c*1'	47 s
200	kodieren('aa') → 'a*2'	46 s
205	kodieren('bbbbba') → 'b*5-a*1'	47 s
210	kodieren('ababbba') → 'a*1 b*1 a*1 b*3 a*1'	45 s
215	kodieren('acaaa') → 'a*1:c*1:a*3'	45 s
220	kodieren('bab') → 'b*1 a*1 b*1'	46 s
225	kodieren('aaccacb') → 'a*2:c*2:a*1:c*1:b*1'	48 s
230	kodieren('bcaabaacab') → 'b*1 c*1 a*2 b*1 a*2 c*1 a*1 b*1'	46 s
235	kodieren('aabbbaac') → 'a*2:b*3:a*1:c*1'	43 s
240	kodieren('abbab') → 'a*1:b*2:a*1:b*1'	44 s
245	kodieren('ba') → 'b*1:a*1'	55 s

250	<code>kodieren('b') → 'b*1'</code>	44 s
255	<code>kodieren('caabac') → 'c*1:a*2:b*1:a*1:c*1'</code>	56 s
260	<code>kodieren('cc') → 'c*2'</code>	44 s
265	<code>kodieren('bca') → 'b*1:c*1:a*1'</code>	44 s
270	<code>kodieren('abcaccbbb') → 'a*1-b*1-c*1-a*1-c*2-b*3'</code>	49 s
275	<code>kodieren('acbcccabaaba') → 'a*1 c*1 b*1 c*3 a*1 b*1 a*2 b*1 ...</code>	44 s
280	<code>kodieren('bbbccbba') → 'b*3-c*2-b*2-a*1'</code>	48 s
285	<code>kodieren('ccabbab') → 'c*2:a*1:b*2:a*1:b*1'</code>	45 s
290	<code>kodieren('b') → 'b*1'</code>	47 s
295	<code>kodieren('aab') → 'a*2-b*1'</code>	43 s
300	<code>kodieren('bcabcaba') → 'b*1-c*1-a*1-b*1-c*1-a*1-b*1-a*1'</code>	57 s
305	<code>kodieren('bbbaabaa') → 'b*3 a*2 b*1 a*2'</code>	42 s
310	<code>kodieren('bbcb') → 'b*2-c*1-b*2'</code>	44 s
315	<code>kodieren('bbaaa') → 'b*2-a*3'</code>	51 s
320	<code>kodieren('bca') → 'b*1-c*1-a*1'</code>	45 s
325	<code>kodieren('cbc') → 'c*1:b*1:c*1'</code>	43 s
330	<code>kodieren('acccc') → 'a*1 c*4'</code>	62 s
335	<code>kodieren('bbababbab') → 'b*2:a*1:b*1:a*1:b*2:a*1:b*1'</code>	52 s
340	<code>kodieren('cbaaaa') → 'c*1 b*1 a*4'</code>	47 s
345	<code>kodieren('baba') → 'b*1:a*1:b*1:a*1'</code>	99 s
350	<code>kodieren('bbb') → 'b*3'</code>	52 s
355	<code>kodieren('ba') → 'b*1:a*1'</code>	45 s
360	<code>kodieren('babaacbbca') → 'b*1:a*1:b*1:a*2:c*1:b*2:c*1:a*1'</code>	44 s
365	<code>kodieren('babbacaba') → 'b*1-a*1-b*2-a*1-c*1-a*1-b*1-a*1'</code>	44 s
370	<code>kodieren('c') → 'c*1'</code>	49 s
375	<code>kodieren('aaaab') → 'a*4-b*1'</code>	44 s

Alle Teiler einer Zahl addieren.

DER HAKEN

Ob die Zahl selbst mitzählt, wechselt je Variante — dieselbe Aufgabe verlangt einmal das eine und einmal das andere Ergebnis.

Alle 62 einzeln. Hinter jeder Nummer steht ihr erster Testfall – daran sieht man, dass es verschiedene Aufgaben sind und nicht dieselbe 62 mal. Der Code jeder einzelnen Variante steht auf der Webseite; auf Papier würde er den Umfang vervielfachen, ohne etwas zu zeigen, was die Geschwister nicht auch zeigen.

071	teilersumme(266) → 480	54 s
076	teilersumme(1692) → 2676	68 s
081	teilersumme(1693) → 1	53 s
086	teilersumme(658) → 1152	44 s
091	teilersumme(1547) → 2016	63 s
096	teilersumme(412) → 316	47 s
101	teilersumme(2467) → 1	48 s
106	teilersumme(2632) → 5760	48 s
111	teilersumme(2433) → 3248	51 s
116	teilersumme(3084) → 7224	51 s
121	teilersumme(1914) → 2406	50 s
126	teilersumme(2365) → 3168	49 s
131	teilersumme(3079) → 3080	51 s
136	teilersumme(1943) → 97	66 s
141	teilersumme(864) → 1656	59 s
146	teilersumme(1806) → 2418	55 s
151	teilersumme(2839) → 185	54 s
156	teilersumme(2404) → 1810	67 s
161	teilersumme(1398) → 1410	53 s
166	teilersumme(2400) → 7812	46 s
171	teilersumme(2033) → 2160	74 s
176	teilersumme(1943) → 97	53 s
181	teilersumme(1111) → 113	80 s
186	teilersumme(1218) → 2880	44 s
191	teilersumme(3246) → 3258	51 s
196	teilersumme(992) → 2016	46 s
201	teilersumme(2076) → 2796	41 s
206	teilersumme(3983) → 4560	58 s
211	teilersumme(1984) → 2080	44 s
216	teilersumme(485) → 103	68 s
221	teilersumme(1033) → 1	65 s
226	teilersumme(1481) → 1482	40 s
231	teilersumme(2606) → 1306	58 s
236	teilersumme(662) → 996	47 s
241	teilersumme(1603) → 237	45 s
246	teilersumme(2392) → 5040	60 s

251	teilersumme(1735) → 2088	51 s
256	teilersumme(598) → 410	44 s
261	teilersumme(3967) → 3968	48 s
266	teilersumme(1857) → 2480	47 s
271	teilersumme(1601) → 1	58 s
276	teilersumme(3341) → 3612	50 s
281	teilersumme(263) → 1	66 s
286	teilersumme(577) → 1	50 s
291	teilersumme(3868) → 2908	56 s
296	teilersumme(3249) → 1704	51 s
301	teilersumme(2687) → 2688	78 s
306	teilersumme(2740) → 3056	46 s
311	teilersumme(698) → 1050	52 s
316	teilersumme(3044) → 5334	53 s
321	teilersumme(3438) → 4050	48 s
326	teilersumme(2448) → 4806	46 s
331	teilersumme(1619) → 1	65 s
336	teilersumme(385) → 191	47 s
341	teilersumme(2687) → 2688	65 s
346	teilersumme(1193) → 1	50 s
351	teilersumme(1901) → 1	49 s
356	teilersumme(2443) → 2800	49 s
361	teilersumme(2059) → 2160	63 s
366	teilersumme(1068) → 1452	53 s
371	teilersumme(3171) → 4864	80 s
376	teilersumme(3057) → 1023	61 s

Das grösste Element in jedem Abschnitt fester Breite finden, während der Abschnitt über die Liste wandert.

DER HAKEN

Ist die Liste kürzer als das Fenster, ist das ein Fehler und kein leeres Ergebnis. Die Fensterbreite wechselt je Variante.

Alle 61 einzeln. Hinter jeder Nummer steht ihr erster Testfall – daran sieht man, dass es verschiedene Aufgaben sind und nicht dieselbe 61 mal. Der Code jeder einzelnen Variante steht auf der Webseite; auf Papier würde er den Umfang vervielfachen, ohne etwas zu zeigen, was die Geschwister nicht auch zeigen.

072	fenstermaxima([8, -22, 23, 23, 11, 29]) → [23, 23, 29]	47 s
077	fenstermaxima([23, -26, -36, -28, 29]) → [23, 29]	86 s
082	fenstermaxima([-30, 27, -10, -23, -23, -19, -2, 12, 13]) → [27, 27, -2, 12, 13]	50 s
087	fenstermaxima([-10, -1, 1, -7, -34, -24, -9, 8, -11, 6, -... → [1, 1, 1, 8, 8, 8, 8, 8]	55 s
092	fenstermaxima([8, 25]) → [25]	51 s
097	fenstermaxima([-28, -8, -40, 23, -38, 1, 23, 32, 9]) → [23, 23, 23, 23, 32, 32]	47 s
102	fenstermaxima([21, -15, -14, -12]) → [21, -14, -12]	55 s
107	fenstermaxima([-7, 7, -30, 18]) → [7, 7, 18]	61 s
112	fenstermaxima([-28, 15, -17, -23]) → [15]	48 s
117	fenstermaxima([37, -38, 20, 9, 19, 3, 34]) → [37, 20, 20, 34]	47 s
122	fenstermaxima([-16, -13, 0, 37, -31, 38, 4]) → [-13, 0, 37, 37, 38, 38]	67 s
127	fenstermaxima([1, 27, 39, -11, 9]) → [39, 39, 39]	76 s
132	fenstermaxima([1, 8, -29]) → [8, 8]	48 s
137	fenstermaxima([8, -1, -22, -29, 3, 9, -36]) → [8, 9, 9]	52 s
142	fenstermaxima([-40, -35, -29, 37, -10, 8, -13]) → [-35, -29, 37, 37, 8, 8]	58 s
147	fenstermaxima([36, 30, 2, -39, 15, -1]) → [36, 30, 15]	52 s
152	fenstermaxima([8, -28, 31, -2, -29, -10, 38, 12, -11, -10... → [31, 31, 31, -2, 38, 38, 38, 12, ...	67 s
157	fenstermaxima([25, 22, -5, 28, 8, -29, -36, 26, 12]) → [28, 28, 28, 28, 26, 26]	47 s
162	fenstermaxima([-7, 9, 15, 0, 36, -40]) → [36, 36]	45 s
167	fenstermaxima([7, -23, 20, -37, 35, -7]) → [20, 20, 35, 35]	43 s
172	fenstermaxima([4, 28, 27, -2, -22, -31, 37, -7, 8, -22]) → [28, 28, 27, -2, -22, 37, 37, 8, ...	57 s
177	fenstermaxima([-5, 18, 18, -1, -4]) → [18, 18, 18]	50 s
182	fenstermaxima([-39, -4, -11, -10]) → [-4, -4, -10]	66 s
187	fenstermaxima([-18, -37, -39, 16, 31, 5]) → [-18, 16, 31, 31]	60 s
192	fenstermaxima([-18, 10, -20, -16, -35, 37, -40]) → [10, 10, -16, -16, 37, 37]	47 s
197	fenstermaxima([-1, 23, 36, -33, 34, 6, 3, 8, -13, -25, 4,... → [36, 36, 36, 34, 34, 8, 8, 10]	54 s
202	fenstermaxima([37, -16, -33, 32, 1, -28, 30]) → [37, 32, 32, 32, 30]	92 s
207	fenstermaxima([-34, 27, -3, 10, -35, 22, 9, 31, 29, -25, ... → [27, 27, 22, 22, 31, 31, 31, 32]	56 s
212	fenstermaxima([-24, 3, 14, 33, -14, 33, 35, 20, -22, -29]) → [14, 33, 33, 33, 35, 35, 35, 20]	49 s
217	fenstermaxima([11, -32, -30, 25, -30, -25, -28, 24, 3, 14... → [11, 25, 25, 25, -25, 24, 24, 24,...	59 s
222	fenstermaxima([28, -25, 34, 20, -40, -5, 38, -28, 28, -14... → [34, 34, 34, 38, 38, 38, 38, 28]	48 s
227	fenstermaxima([-21, 15, -27, 9, 22, -25, 8, -5]) → [15, 15, 9, 22, 22, 8, 8]	54 s
232	fenstermaxima([4, 14, 6, 35, 39, 13, -22, 30, -27]) → [39, 39, 39, 39, 39]	53 s
237	fenstermaxima([13, -39, 18, -31, 4, 34, 24, 6]) → [18, 34, 34, 34]	47 s

242	fenstermaxima([8, 36, -30, -18, -16, 16, -30, 26]) → [36, 36, -16, 16, 16, 26]	56 s
247	fenstermaxima([12, -12, -37, 37, 18, 39, 28, -1, 37, -22,... → [37, 39, 39, 39, 39, 39, 37, 37]	54 s
252	fenstermaxima([26, -12, 32, -26]) → [26, 32, 32]	46 s
257	fenstermaxima([15, 14, 31, 27, 21, 17]) → [15, 31, 31, 27, 21]	97 s
262	fenstermaxima([-20, 14, 39, -26, 16, 39]) → [39, 39]	49 s
267	fenstermaxima([-9, 0, 3, -8, -9, 18, 38, -8]) → [3, 18, 38, 38]	46 s
272	fenstermaxima([-39, -19, 32, -4, -8, 6, 0, 14, -16, -13, ... → [32, 32, 32, 6, 6, 14, 14, 14, 16]	51 s
277	fenstermaxima([19, 20, -23, -14, 37]) → [37]	63 s
282	fenstermaxima([-27, 28, -35, -14, -13, 2, -32]) → [28, 28, 2, 2]	48 s
287	fenstermaxima([3, -21, -7, -29]) → [3, -7, -7]	50 s
292	fenstermaxima([6, -35, 33, 19, -33, 35, -17]) → [33, 33, 35, 35]	47 s
297	fenstermaxima([0, 37, -36, -21, 37, 5, -28, -40, -9]) → [37, 37, 37, 37, 37]	53 s
302	fenstermaxima([-26, 0, -32, 28, -6, 31]) → [28, 28, 31]	46 s
307	fenstermaxima([-30, -40, -23, 16, -36, 0, -1, 15, 14]) → [-30, -23, 16, 16, 0, 0, 15, 15]	67 s
312	fenstermaxima([19, 1, 16, 0, -33, 22, 10]) → [19, 22, 22]	57 s
317	fenstermaxima([6, -12, -35, 28, -24, -4, 26, 8, -29]) → [28, 28, 28, 28, 26, 26]	54 s
322	fenstermaxima([28, -26, 28, -28, -13, -6, 38, 6]) → [28, 28, 28, -6, 38, 38]	55 s
327	fenstermaxima([-20, -39, 31, 6, -18, -15, -22, -17, 2, -2... → [31, 31, 31, 6, -15, -15, 2, 2]	46 s
332	fenstermaxima([-24, 13, -26, 2, 5, 25, 23]) → [13, 13, 2, 5, 25, 25]	53 s
337	fenstermaxima([5, -21, -28, 33, 29, 2, -18]) → [5, 33, 33, 33, 29]	51 s
342	fenstermaxima([-35, -3, -4, -32, -33, 34, 23, -29]) → [-3, -3, -4, -32, 34, 34, 23]	54 s
347	fenstermaxima([-25, 14, 8, 38, -18]) → [38]	49 s
352	fenstermaxima([32, -11, 37, 1, 1, 17, -32, -7]) → [32, 37, 37, 1, 17, 17, -7]	58 s
357	fenstermaxima([-35, 12, -36, 18, 21, -10, -26, 18, 16]) → [12, 12, 18, 21, 21, -10, 18, 18]	56 s
362	fenstermaxima([-9, -9, 21, -1, -4]) → [21, 21, 21]	52 s
367	fenstermaxima([11, 26, 13, 14, -3, -3, -24, -16, 28, -27,... → [26, 26, 14, 14, -3, 28, 28, 28, ...	212 s
372	fenstermaxima([-11, 33, 26]) → [33, 33]	51 s

Eine Zahl in einer anderen Basis darstellen.

DER HAKEN

Die Basis wechselt je Variante, ebenso der erlaubte Zeichenvorrat. Null und negative Zahlen brauchen jedes Mal eine eigene Entscheidung.

Alle 61 einzeln. Hinter jeder Nummer steht ihr erster Testfall – daran sieht man, dass es verschiedene Aufgaben sind und nicht dieselbe 61 mal. Der Code jeder einzelnen Variante steht auf der Webseite; auf Papier würde er den Umfang vervielfachen, ohne etwas zu zeigen, was die Geschwister nicht auch zeigen.

073	<code>in_basis(406) → '1514'</code>	52 s
078	<code>in_basis(613) → '431'</code>	55 s
083	<code>in_basis(9033) → '5289'</code>	63 s
088	<code>in_basis(7021) → '211041'</code>	47 s
093	<code>in_basis(2280) → '14320'</code>	48 s
098	<code>in_basis(304) → '102021'</code>	52 s
103	<code>in_basis(32901) → '412153'</code>	53 s
108	<code>in_basis(3) → '3'</code>	54 s
113	<code>in_basis(53) → '104'</code>	62 s
118	<code>in_basis(9) → '14'</code>	48 s
123	<code>in_basis(75) → '63'</code>	50 s
128	<code>in_basis(9) → '12'</code>	50 s
133	<code>in_basis(15173) → '62144'</code>	50 s
138	<code>in_basis(9257) → '35663'</code>	54 s
143	<code>in_basis(911) → '12121'</code>	50 s
148	<code>in_basis(2) → '2'</code>	54 s
153	<code>in_basis(11) → '14'</code>	59 s
158	<code>in_basis(8) → '11'</code>	51 s
163	<code>in_basis(366) → '266'</code>	51 s
168	<code>in_basis(33028) → '1200022021'</code>	47 s
173	<code>in_basis(8) → '22'</code>	50 s
178	<code>in_basis(95) → '164'</code>	66 s
183	<code>in_basis(6) → '11'</code>	48 s
188	<code>in_basis(46) → '3a'</code>	54 s
193	<code>in_basis(48) → '40'</code>	53 s
198	<code>in_basis(0) → '0'</code>	71 s
203	<code>in_basis(6786) → '3b16'</code>	50 s
208	<code>in_basis(0) → '0'</code>	58 s
213	<code>in_basis(17) → '23'</code>	62 s
218	<code>in_basis(62) → '116'</code>	57 s
223	<code>in_basis(77) → '205'</code>	56 s
228	<code>in_basis(8) → '22'</code>	57 s
233	<code>in_basis(4129) → '12122221'</code>	51 s
238	<code>in_basis(1179) → '14204'</code>	52 s
243	<code>in_basis(8) → '12'</code>	53 s
248	<code>in_basis(351) → '2401'</code>	48 s

253	<code>in_basis(660) → '470'</code>	50 s
258	<code>in_basis(4750) → '123000'</code>	48 s
263	<code>in_basis(282) → '1150'</code>	67 s
268	<code>in_basis(8133) → '101353'</code>	55 s
273	<code>in_basis(40224) → '2241344'</code>	60 s
278	<code>in_basis(237) → '456'</code>	55 s
283	<code>in_basis(781) → '2164'</code>	54 s
288	<code>in_basis(80) → '68'</code>	59 s
293	<code>in_basis(7280) → '30140'</code>	51 s
298	<code>in_basis(14) → '24'</code>	55 s
303	<code>in_basis(3) → '3'</code>	57 s
308	<code>in_basis(555) → '3a3'</code>	48 s
313	<code>in_basis(4) → '4'</code>	64 s
318	<code>in_basis(3415) → '1b87'</code>	56 s
323	<code>in_basis(28340) → '14498'</code>	51 s
328	<code>in_basis(16338) → '9556'</code>	50 s
333	<code>in_basis(4) → '4'</code>	50 s
338	<code>in_basis(7) → '10'</code>	51 s
343	<code>in_basis(7) → '7'</code>	43 s
348	<code>in_basis(22) → '34'</code>	104 s
353	<code>in_basis(3) → '3'</code>	62 s
358	<code>in_basis(95) → '164'</code>	63 s
363	<code>in_basis(6) → '6'</code>	47 s
368	<code>in_basis(56978) → '2220011022'</code>	50 s
373	<code>in_basis(41) → '56'</code>	53 s

Was diese Seite nicht zeigt

Das sind kleine, klar umrissene Aufgaben. Eine Datei, eine Funktion, ein Test, der genau sagt was richtig ist. Das ist etwas völlig anderes als Arbeit in einem gewachsenen System mit unklaren Anforderungen.

308 der 376 Aufgaben sind Varianten aus 5 Familien. Der Lauf belegt Ausdauer über sechs Stunden — die Bandbreite belegt eher der Teil mit den 68 verschiedenen Aufgaben.

Ein späterer Lauf über zwölf Stunden ist durchgefallen. Der Agent schrieb eine Endlosschleife und bemerkte sie auch im zweiten Anlauf nicht. Sechs saubere Stunden heissen nicht, dass es immer sauber läuft.

Und ein Teil der Fehler lag am Prüfstand, nicht am Agenten. Die erzeugten Tests riefen Funktionen in einer Form auf, die nach variabler Argumentzahl aussah — worauf der Agent die vorgegebene Schnittstelle umschrieb. Wer Agenten misst, misst zuerst sein eigenes Messgerät.

Modell: Qwen3.8-27B, lokal auf einer NVIDIA GB10 · Agentenrahmen: Hermes
6.0 Stunden reine Aufgabenzeit · Median 52 s je Aufgabe
Alle Zahlen stammen aus der Laufdatenbank, keine ist geschätzt.